DEVOPS ENGINEERING MASTERY

CI/CD PIPELINE ENGINEERING

Continuous Integration, Continuous Delivery, Deployment Automation & Release Engineering

VOLUME 05 • FIRST EDITION

```
nahid@technova:~$ gh workflow run ci.yml
✓ build  •  ✓ test  •  ✓ lint  •  ✓ docker build
nahid@technova:~$ docker push ghcr.io/technova/app:v1.4.0
✓ image pushed — digest sha256:9f2c…
nahid@technova:~$ kubectl rollout status deployment/technova-app
```

GitHub Actions • GitLab CI/CD • Jenkins • Docker • MERN

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

CI/CD Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 05 (Companion Workbook). First Edition, 8 November 2026. GitHub Actions, GitLab CI/CD, Jenkins and Docker with a MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your repository without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab step on a production pipeline. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

ল্যাব ওয়ার্কবুক

ল্যাব ওয়ার্কবুক — CI/CD Pipeline Engineering

এই ওয়ার্কবুকটি **DevOps Engineering Mastery — Volume 05: CI/CD Pipeline Engineering** বইয়ের হাতে-কলমে অংশ। এখানে প্রতিটি ল্যাব একটি বাস্তব সফটওয়্যার ডেলিভারি কাজের ছোট সংস্করণ। পড়ে শুধু জানা যাবে না — pipeline লিখতে হবে, ভাঙতে হবে, ঠিক করতে হবে, আবার চালাতে হবে।

পরিবেশ (Environment): Node.js 20 LTS, Git, GitHub অ্যাকাউন্ট, GitLab অ্যাকাউন্ট (gitlab.com), Docker Desktop অথবা Linux-এ Docker Engine, এবং একটি MERN নমুনা অ্যাপ (Express API + React frontend + MongoDB)। সব ল্যাব একটি ব্যক্তিগত (private) টেস্ট রিপোজিটরিতে করবেন — কোনো লাইভ প্রোডাকশন রিপোজিটরি বা প্রোডাকশন সার্ভারে নয়।

> **সতর্কতা — ল্যাব বনাম প্রোডাকশন:** ল্যাবে secret, token বা credential ব্যবহার করলে সবসময় কাল্পনিক মান ব্যবহার করুন — যেমন `CHANGE-ME-SECRET`। আসল API key, পাসওয়ার্ড বা token কখনো কোডে, স্ক্রিনশটে বা চ্যাটে দেবেন না। Secret কখনো `console.log` বা pipeline log-এ ছাপাবেন না। Self-hosted runner-এ অচেনা pull request-এর কোড চালাবেন না — সেটি আপনার মেশিনের secret চুরি করতে পারে।

> **কীভাবে এই ওয়ার্কবুক ব্যবহার করবেন:** ল্যাবগুলো ক্রমানুসারে করুন — প্রতিটি ল্যাব আগের ল্যাবের উপর দাঁড়িয়ে আছে। Configuration কপি করার আগে প্রতিটি field-এর ব্যাখ্যা পড়ুন। প্রতিটি ল্যাবের শেষে Verification ধাপ দিয়ে ফল যাচাই করুন, তারপর Reflection প্রশ্নের উত্তর নিজে লিখুন। Cleanup ধাপটি এড়িয়ে যাবেন না — অপ্রয়োজনীয় runner, image ও environment মুছে ফেলা অপারেশনেরই অংশ।

সাধারণ নিয়ম

বিষয়	নিয়ম
ডোমেইন	কাল্পনিক ডোমেইন ব্যবহার করুন: `technova.example`, `app.technova.example`, `staging.technova.example`। আসল ডোমেইন ব্যবহার করবেন না।
Secret	উদাহরণে `CHANGE-ME-SECRET` লিখুন। আসল secret কখনো commit করবেন না; `.gitignore`-এ `.env` রাখুন।
আউটপুট	

	সব নমুনা টার্মিনাল ও pipeline আউটপুট চিত্রণমূলক (illustrative) — লেখা থাকবে “নমুনা আউটপুট — চিত্রণমূলক”। আপনার সিস্টেমে সময়, commit hash ও সংস্করণ আলাদা হতে পারে, সেটাই স্বাভাবিক।
Branch	মূল branch: `main`। Feature branch: `feature/<নাম>`। Release tag: `v1.2.3` (Semantic Versioning)।
নমুনা অ্যাপ	TechNova MERN অ্যাপ — `server/` (Express + MongoDB), `client/` (React/Vite), `Dockerfile`, `package.json`। না থাকলে ল্যাব ২-এর ন্যূনতম অ্যাপ দিয়ে শুরু করুন।

প্রতিটি ল্যাবে ব্যবহৃত ইংরেজি টেকনিক্যাল শব্দ প্রথমবার বাংলায় ব্যাখ্যা করা হয়েছে। যেমন — **pipeline** হলো কোড commit থেকে deployment পর্যন্ত ধাপগুলোর স্বয়ংক্রিয় ধারাবাহিকতা; **artifact** হলো build-এর ফল — যেমন Docker image বা zip ফাইল — যা পরে deploy করা হয়; **runner** হলো সেই মেশিন বা agent যা pipeline-এর job চালায়।

ল্যাবগুলোর মানচিত্র

ল্যাব ১–৬: CI-এর ভিত্তি ও GitHub Actions পরিচিতি। ল্যাব ৭–১৩: পুনঃব্যবহার, matrix, cache, artifact, secret ও OIDC। ল্যাব ১৪–১৬: GitLab CI/CD। ল্যাব ১৭–১৯: Jenkins। ল্যাব ২০–২৪: টেস্ট, Docker, নিরাপত্তা, staging approval, verification ও rollback — অর্থাৎ সম্পূর্ণ delivery chain।

ল্যাব 1

Manual Deployment বনাম Automated Delivery তুলনা করা

উদ্দেশ্য (Objective): একই MERN অ্যাপ একবার হাতে (manual) ও একবার একটি সরল script দিয়ে deploy করে সময়, ভুলের ঝুঁকি ও পুনরাবৃত্তিযোগ্যতার (repeatability) পার্থক্য নিজে মেপে দেখা। এই ল্যাবের পর আপনি বুঝবেন কেন CI/CD শুধু "সুবিধা" নয়, বরং নির্ভরযোগ্য delivery-র পূর্বশর্ত।

পূর্বশর্ত (Prerequisites): Git ও Node.js 20 ইনস্টল আছে; একটি ছোট Express অ্যাপ (`server.js` — "Hello TechNova" দেয়) চালাতে পারেন। **প্রয়োজনীয় টুল (Tools):** Git, Node.js 20, টার্মিনাল, ঘড়ি/টাইমার, একটি নোট ফাইল।

আর্কিটেকচার:

Manual পথ: Developer → SSH → সার্ভার → git pull → npm install → npm run build → pm2 restart (প্রতিটি ধাপ হাতে, ভুলের সুযোগ প্রতিটি ধাপে)

Automated পথ: Developer → git push → Pipeline → build → test → deploy (একই ধাপ, একই ক্রম, প্রতিবার)

সম্পূর্ণ কনফিগারেশন

```
# deploy.sh — হাতে করা ধাপগুলোকে একটি script-এ বাঁধা (প্রথম automation)
#!/usr/bin/env bash
set -euo pipefail
echo "Deploying TechNova..."
git pull origin main
npm ci
npm test
npm run build
echo "Build OK — restart step would run here (lab-এ শুধু echo)"
# নমুনা আউটপুট — চিত্রণমূলক
# Deploying TechNova... Build OK
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: Express অ্যাপটি তৈরি করুন ও `node server.js` দিয়ে স্থানীয়ভাবে চালান। **ধাপ ২:** Manual deploy অনুকরণ করুন: টার্মিনালে একে একে `git pull`, `npm install`, `npm test`, `npm run build` চালান — প্রতিটি ধাপের সময় লিখে রাখুন। **ধাপ ৩:** ইচ্ছা করে একটি ভুল করুন — যেমন `npm test` বাদ দিয়ে সরাসরি build করুন — এবং লিখুন কী ঝুঁকি তৈরি হলো। **ধাপ ৪:** উপরের `deploy.sh` তৈরি করুন, `chmod +x deploy.sh` দিন, চালান। **ধাপ ৫:** Script-এ একটি ধাপ ব্যর্থ করুন (যেমন test-এ ইচ্ছা করে ভুল assert) — দেখুন `set -e`-এর কারণে script সেখানেই থামে, deploy এগোয় না। **ধাপ ৬:** দুই পদ্ধতির সময়, ধাপের সংখ্যা ও ভুলের সম্ভাবনা একটি টেবিলে লিখুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
bash deploy.sh
# প্রত্যাশিত: সব ধাপ সফল হলে "Build OK" দেখাবে; কোনো ধাপ ব্যর্থ হলে
# script সেখানেই থেমে যাবে এবং deploy এগোবে না — এটাই fail-fast আচরণ।
```

প্রত্যাশিত ফলাফল: Manual পদ্ধতিতে সময় বেশি ও ধাপ বাদ যাওয়ার ঝুঁকি স্পষ্ট; script পদ্ধতিতে একই ক্রম প্রতিবার, এবং ব্যর্থ ধাপে deploy থেমে যায়। আপনার নোটে দুই পদ্ধতির তুলনা টেবিল থাকবে।

সাধারণ ব্যর্থতা ও সমাধান

Script চলছে না (permission denied): `chmod +x deploy.sh` দিন। **`npm ci` ব্যর্থ:** `package-lock.json` আছে কি না দেখুন; না থাকলে প্রথমবার `npm install` করুন। **Test ব্যর্থ হলেও deploy এগোচ্ছে:** `set -e` লাইনটি script-এর শুরুতে আছে কি না দেখুন — এটাই fail-fast-এর চাবি।

Cleanup

`node_modules/` ও build আউটপুট মুছুন (`rm -rf node_modules dist`)। `deploy.sh` রিপোজিটরিতে রাখুন — পরের ল্যাবগুলোতে এটিই pipeline-এর ভিত্তি হবে।

Reflection প্রশ্ন

১. Manual deploy-এ কোন ধাপটি বাদ গেলে সবচেয়ে বড় ক্ষতি হয়, এবং কেন? ২. `set -euo pipefail` না থাকলে ব্যর্থ test-ও deploy হতে পারত — এর প্রোডাকশন ঝুঁকি কী? ৩. Script কি CI/CD-র সমান? না হলে আর কী কী দরকার (trigger, ইতিহাস, অনুমোদন)?

ল্যাব 2

একটি মৌলিক CI Workflow তৈরি করা

উদ্দেশ্য (Objective): GitHub Actions-এ প্রথম CI workflow লেখা — প্রতিটি push-এ Node.js অ্যাপ install, lint ও test স্বয়ংক্রিয়ভাবে চালানো। এটাই CI-এর মূল: প্রতিটি পরিবর্তন merge-এর আগেই যাচাই করা।

পূর্বশর্ত (Prerequisites): ল্যাব ১ সম্পূর্ণ; GitHub অ্যাকাউন্ট ও একটি private টেস্ট রিপোজিটরি আছে; Node.js অ্যাপে `package.json`-এ `test` ও `lint` script আছে। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
git push → GitHub → Event: push → Workflow: ci.yml → Runner (ubuntu-latest) → Steps: checkout → setup-node → npm ci → npm run lint → npm test → ✅/❌ Status check

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
permissions:
  contents: read
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

Field ব্যাখ্যা: `on` — কখন workflow চলবে (trigger); `permissions: contents: read` — least privilege, workflow শুধু কোড পড়তে পারবে; `runs-on: ubuntu-latest` — GitHub-hosted runner; `actions/checkout@v4` — কোড runner-এ নামায়; `setup-node` — Node.js 20 বসায় ও npm cache চালু করে; `npm ci` — lockfile অনুযায়ী নির্ভুল install।

ধাপে ধাপে নির্দেশনা

ধাপ ১: রিপোজিটরিতে `.github/workflows/ci.yml` তৈরি করুন। **ধাপ ২:** ন্যূনতম অ্যাপ না থাকলে তৈরি করুন: `npm init -y`, `npm install express`, `package.json`-এ `"test": "node --test"` ও `"lint": "node --check server.js"` যোগ করুন। **ধাপ ৩:** Commit ও push করুন। **ধাপ ৪:** GitHub-এ Actions ট্যাবে গিয়ে workflow run দেখুন

— প্রতিটি step সবুজ কি না। **ধাপ ৫:** ইচ্ছা করে test ভাঙুন (একটি assert ভুল করুন), push করুন — workflow লাল হওয়া দেখুন। **ধাপ ৬:** ঠিক করে আবার push করুন — সবুজ ফিরে আসা দেখুন।

যাচাই ও প্রত্যাশিত ফলাফল

Actions ট্যাবে run-এর status ✅ সবুজ, প্রতিটি step-এর log-এ `npm ci`, lint ও test সফল। ভাঙা test-এ run ❌ লাল — এটাই প্রমাণ যে CI ভাঙা কোড ধরতে পারে।

সাধারণ ব্যর্থতা ও সমাধান

Workflow চলছে না: ফাইলটি ঠিক `.github/workflows/`-এ আছে কি না, YAML indentation ঠিক কি না দেখুন। **`npm ci` ব্যর্থ:** `package-lock.json` commit করা হয়েছে কি না দেখুন। **Action version সতর্কতা:** `@v4` পিন করা আছে কি না দেখুন — ভাসমান (unpinned) action ব্যবহার করবেন না।

Cleanup

টেস্টের ভাঙা commit-টি ইতিহাসে থাকতে পারে — ল্যাব রিপোজিটরি বলে সমস্যা নেই, তবে শেয়ার্ড রিপোজিটরিতে ভাঙা commit push করবেন না। Workflow ফাইলটি রেখে দিন — ল্যাব ৩-এ এটিই বাড়ানো হবে।

Reflection প্রশ্ন

১. `npm ci` আর `npm install`-এর পার্থক্য কী, এবং CI-তে `ci` কেন ভালো? ২. `permissions: contents: read` না দিলে কী ঝুঁকি থাকে? ৩. Push-এ CI চলা আর pull request-এ CI চলার উদ্দেশ্য কি এক?

ল্যাব 3

Pull Request যাচাই (Validate Pull Requests)

উদ্দেশ্য (Objective): Pull request (PR)-এ স্বয়ংক্রিয় যাচাই বসানো — CI সবুজ না হলে merge বন্ধ থাকবে। এতে ভাঙা কোড `main`-এ ঢুকতে পারে না, এবং reviewer-রা কোডের গুণমান নিয়ে কথা বলতে পারেন, ভাঙা build নিয়ে নয়।

পূর্বশর্ত (Prerequisites): ল্যাব ২ সম্পূর্ণ; CI workflow আছে। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
Feature branch → Pull Request → GitHub: CI workflow চলে → Status check (✅/❌) → Branch protection: CI পাস না হলে Merge button বন্ধ → Review + Merge → main সুরক্ষিত থাকে

সতর্কতা: এই ল্যাবে branch protection চালু করবেন শুধু নিজের টেস্ট রিপোজিটরিতে। শেয়ার্ড টিম রিপোজিটরিতে নিয়ম বদলানোর আগে টিমের অনুমতি নিন।

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/ci.yml — pull_request trigger আগেই আছে (ল্যাব ২ দেখুন)
# Branch protection (GitHub UI): Settings → Branches → Add rule → main
#   ☑ Require a pull request before merging
#   ☑ Require status checks to pass before merging → CI / build নির্বাচন করুন
#   ☑ Require branches to be up to date before merging (ঐচ্ছিক, ল্যাবে চালু করুন)
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: `git checkout -b feature/greeting` তৈরি করুন, `server.js`-এ একটি ছোট পরিবর্তন করুন, commit ও push করুন। **ধাপ ২:** GitHub-এ Pull Request খুলুন (feature/greeting → main)। **ধাপ ৩:** PR পাতার নিচে CI check চলতে দেখুন — হলুদ (চলছে) → সবুজ (পাস)। **ধাপ ৪:** Branch protection চালু করুন (উপরের কনফিগারেশন দেখুন)। **ধাপ ৫:** আবার একটি ভাঙা পরিবর্তন (ভুল test) দিয়ে PR খুলুন — Merge button বন্ধ থাকা দেখুন। **ধাপ ৬:** Test ঠিক করে push করুন — একই PR-এ CI আবার চলে, সবুজ হলে merge করুন।

যাচাই ও প্রত্যাশিত ফলাফল

ভালো PR-এ “All checks have passed” ও সবুজ Merge button; ভাঙা PR-এ “Some checks were not successful” ও বন্ধ Merge button। Merge-এর পর `main`-এ শুধু যাচাই করা কোড থাকে।

সাধারণ ব্যর্থতা ও সমাধান

Status check তালিকায় আসছে না: প্রথমে একটি PR-এ CI অন্তত একবার চলতে হবে, তারপর check-এর নাম তালিকায় আসে। **Merge এখনো খোলা:** Rule-টি `main` branch-এ প্রয়োগ হয়েছে কি না, এবং check-এর নাম ঠিক (`build`) কি না দেখুন। **নিজের PR নিজে merge করতে পারছেন না:** Rule-এ “Require review” চালু থাকলে অন্য reviewer লাগবে — ল্যাবে সেটি বন্ধ রাখুন।

Cleanup

Merge করা feature branch মুছুন (`git branch -d feature/greeting` ও GitHub-এ Delete branch)। Branch protection ল্যাব রিপোজিটরিতে রেখে দিতে পারেন — এটাই সঠিক ডিফল্ট।

Reflection প্রশ্ন

১. CI পাস না হলে merge বন্ধ রাখা কেন “গেট” (quality gate) বলে? ২. “Require branches to be up to date” চালু থাকলে কী সুবিধা ও কী অসুবিধা? ৩. PR-এ CI চালানো আর main-এ push-এ CI চালানো — দুটোর ভূমিকা কি আলাদা?

ল্যাব 4

Branch-ভিত্তিক Trigger কনফিগার করা

উদ্দেশ্য (Objective): Branch ও path অনুযায়ী আলাদা trigger বসানো — `main`-এ push-এ full CI, feature branch-এ হালকা যাচাই, `release/*`-এ deployment প্রস্তুতি। ভুল trigger-এ pipeline অপ্রয়োজনীয়ভাবে চলে, খরচ ও সময় বাড়ে; সঠিক trigger-এ শুধু দরকারি কাজ চলে।

পূর্বশর্ত (Prerequisites): ল্যাব ২–৩ সম্পূর্ণ। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
push to main → full CI (lint + test + build) | push to feature/* → lint + test | push to release/* → build + artifact | docs/ পরিবর্তন → workflow চলবে না (paths-ignore)

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main, 'release/**']
    paths-ignore: ['docs/**', '**.md']
  pull_request:
    branches: [main]
    paths: ['server/**', 'client/**', 'package*.json']
permissions:
  contents: read
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run build
        if: github.ref == 'refs/heads/main'
```

Field ব্যাখ্যা: `branches` — কোন branch-এ trigger হবে; `'release/**'` — glob pattern, `release/v1.2`-এর মতো branch ধরে; `paths-ignore` — শুধু docs বদলালে workflow চলবে না; `paths` — PR-এ শুধু কোড বদলালে চলবে; `if: github.ref == ...` — build step শুধু main-এ চলবে।

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের trigger দিয়ে `ci.yml` আপডেট করুন। **ধাপ ২:** শুধু `README.md` বদলে main-এ push করুন — workflow না চলা দেখুন (paths-ignore কাজ করছে)। **ধাপ ৩:** `feature/trigger-test` branch-এ কোড বদলে PR

খুলুন — CI চলা দেখুন। **ধাপ ৪:** `release/v1.0` branch তৈরি করে push করুন — full CI চলা দেখুন। **ধাপ ৫:** Actions ট্যাবে প্রতিটি run-এর trigger কারণ (event ও branch) মিলিয়ে দেখুন।

যাচাই ও প্রত্যাশিত ফলাফল

Docs-শুধু push-এ কোনো run তৈরি হয় না; কোড push-এ run তৈরি হয়; `release/*` push-এ run চলে; build step শুধু main-এর run-এ চলে। প্রতিটি run-এর শিরোনামে সঠিক branch নাম দেখা যায়।

সাধারণ ব্যর্থতা ও সমাধান

Workflow চলছে না যদিও কোড বদলেছি: `paths` ফিল্টারে ফোল্ডার নাম ঠিক কি না দেখুন — `server/**` মানে `server/`-এর ভেতরের ফাইল। **Docs push-এও চলছে:** `paths-ignore` শুধু `push`-এ আছে, `pull_request`-এ নেই — দুটোতেই যোগ করুন। **Glob কাজ করছে না:** pattern কোটেশনে ('release/**') লেখা আছে কি না দেখুন।

Cleanup

টেস্ট branch-গুলো (`release/v1.0`, `feature/trigger-test`) মুছে ফেলুন। `ci.yml`-এর trigger কনফিগারেশন রেখে দিন।

Reflection প্রশ্ন

১. সব push-এ full pipeline চালালে কী কী খরচ বাড়ে? ২. `paths` ফিল্টার কখন বিপজ্জনক হতে পারে (যেমন শেয়ার্ড config বদলালে)? ৩. Branch trigger আর `if` condition — কখন কোনটি ব্যবহার করবেন?

ল্যাব 5

Build ও Test Job তৈরি করা

উদ্দেশ্য (Objective): একটি workflow-কে দুটি আলাদা job-এ ভাগ করা — `test` job ও `build` job — যেখানে build শুধু test পাস করলে চলে (`needs`)। এতে ব্যর্থ test-এ সময় নষ্ট করে build চলে না, এবং প্রতিটি job-এর দায়িত্ব স্পষ্ট থাকে।

পূর্বশর্ত (Prerequisites): ল্যাব ২ সম্পূর্ণ; `package.json`-এ `build` script আছে। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
Workflow → Job: test (lint + unit test) ── needs ──▶ Job: build (npm run build → dist/ তৈরি) → Job: build ব্যর্থ test-এ শুরুই হয় না (skipped)

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/ci.yml
name: CI
on: { push: { branches: [main] }, pull_request: { branches: [main] } }
permissions: { contents: read }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test
  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm run build
      - run: ls -la dist/ || ls -la build/
```

Field ব্যাখ্যা: `needs: test` — build job test job-এর সফল সমাপ্তির উপর নির্ভরশীল; test ব্যর্থ হলে build `skipped` হয়। প্রতিটি job আলাদা runner-এ চলে — তাই প্রতিটি job-এ আবার checkout ও install লাগে (এই খরচ ল্যাব ১০-এর cache কমায়)।

ধাপে ধাপে নির্দেশনা

ধাপ ১: `package.json`-এ build script যোগ করুন (যেমন `"build": "mkdir -p dist && cp server.js dist/"` — নমুনা অ্যাপের জন্য)। **ধাপ ২:** উপরের দুই-job workflow লিখে push করুন। **ধাপ ৩:** Actions ট্যাবে দুটি job সমান্তরালে নয়, ক্রমানুসারে চলা দেখুন — build শুরু হয় test শেষে। **ধাপ ৪:** Test ভেঙে push করুন — build job `skipped` হওয়া দেখুন। **ধাপ ৫:** Test ঠিক করে push করুন — দুটি job-ই সবুজ হওয়া দেখুন।

যাচাই ও প্রত্যাশিত ফলাফল

সফল run-এ test ✅ তারপর build ✅ ব্যর্থ test-এ test ❌ও build ⏭️skipped। Build job-এর log-এ `dist/` তৈরি হওয়া দেখা যায়।

সাধারণ ব্যর্থতা ও সমাধান

দুটি job একসাথে চলছে: `needs` লেখা হয়েছে কি না দেখুন — না থাকলে job-গুলো সমান্তরাল (parallel) চলে। **Build job-এ ফাইল পাওয়া যাচ্ছে না:** প্রতিটি job নতুন runner — test job-এর ফাইল build job-এ থাকে না; ফাইল পাঠাতে artifact লাগে (ল্যাব ১১)। **`npm run build` নেই:** script-এর নাম `package.json`-এর সঙ্গে মিলিয়ে দেখুন।

Cleanup

`dist/` ফোল্ডারটি `.gitignore`-এ রাখুন যাতে build আউটপুট commit না হয়। Workflow ফাইলটি রেখে দিন।

Reflection প্রশ্ন

১. Job আলাদা করলে কী সুবিধা, আর কী খরচ (প্রতিটি job-এ আবার install)? ২. `needs` ছাড়া দুটি job সমান্তরালে চললে কী সমস্যা হতে পারে? ৩. কোন কাজগুলো একই job-এ রাখা উচিত, কোনগুলো আলাদা job-এ?

ল্যাব 6

GitHub Actions Runner বোঝা ও পরীক্ষা করা

উদ্দেশ্য (Objective): GitHub-hosted runner-এর ভেতরে কী আছে তা নিজে দেখা — OS, ইনস্টল করা টুল, পরিবেশ ভ্যারিয়েবল, কাজের ফোল্ডার — এবং self-hosted runner কখন দরকার তা বোঝা। Runner অদৃশ্য "জাদুর বাক্স" নয়; এর সীমা জানলে pipeline ডিবাগ করা সহজ হয়।

পূর্বশর্ত (Prerequisites): ল্যাব ২ সম্পূর্ণ। **প্রয়োজনীয় টুল:** GitHub, Git; ঐচ্ছিকভাবে একটি ল্যাব VM বা নিজের কম্পিউটার (self-hosted runner পরীক্ষার জন্য)।

আর্কিটেকচার:
GitHub → Runner (ubuntu-latest: Ubuntu VM, প্রি-ইনস্টল টুলসহ) → Workspace: $GITHUB_WORKSPACE → Steps চলে → Run শেষে VM মুছে যায় (ephemeral)
Self-hosted বিকল্প: আপনার মেশিন → Runner agent → একই workflow, আপনার হার্ডওয়্যার ও নেটওয়ার্কে

সতর্কতা: Self-hosted runner-এ public রিপোজিটরির pull request-এর কোড চালাবেন না — অচেনা কোড runner মেশিনের ফাইল ও secret পড়তে পারে। এই ল্যাবের self-hosted অংশ শুধু private টেস্ট রিপোজিটরিতে করবেন।

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/inspect-runner.yml
name: Inspect Runner
on: { workflow_dispatch: {} }
permissions: { contents: read }
jobs:
  inspect:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          echo "OS: $(uname -a)"
          node --version; npm --version; docker --version
          echo "Workspace: $GITHUB_WORKSPACE"
          echo "Runner OS: $RUNNER_OS  Arch: $RUNNER_ARCH"
          ls /usr/local/bin | head -20
          df -h . ; free -h
# Self-hosted runner যোগ করা (GitHub UI):
# Settings → Actions → Runners → New self-hosted runner → Linux → নির্দেশনা অনুযায়ী
# ./config.sh --url https://github.com/<owner>/<repo> --token CHANGE-ME-TOKEN
# ./run.sh   (token-টি একবারের জন্য, কোথাও সংরক্ষণ করবেন না)
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের workflow তৈরি করে Actions ট্যাবে `workflow_dispatch` দিয়ে হাতে চালান। **ধাপ ২:** Log-এ OS, Node, npm, Docker সংস্করণ ও workspace পথ লিখে নিন। **ধাপ ৩:** একটি step-এ ইচ্ছা করে `pwd && ls -la` চালিয়ে দেখুন কোড কোথায় নামে। **ধাপ ৪ (ঐচ্ছিক):** নিজের কম্পিউটারে self-hosted runner বসান, `runs-on: self-hosted`

দিয়ে একটি ছোট workflow চালান, log-এ হোস্টনেম মিলিয়ে দেখুন। **ধাপ ৫:** দুই runner-এর তুলনা লিখুন: শুরু সময়, টুল, খরচ, নিরাপত্তা।

যাচাই ও প্রত্যাশিত ফলাফল

Hosted run-এর log-এ Ubuntu, Node 20, Docker সংস্করণ ও `/home/runner/work/...` পথ দেখা যায়। Self-hosted run-এ আপনার মেশিনের হোস্টনেম ও পথ দেখা যায়। দুটি run-এর মধ্যে workspace পথ আলাদা — এটাই প্রমাণ যে প্রতিটি run আলাদা পরিবেশ।

সাধারণ ব্যর্থতা ও সমাধান

`workflow_dispatch` বোতাম নেই: Workflow ফাইলটি `main`-এ push করা হয়েছে কি না দেখুন — dispatch বোতাম default branch-এর workflow দেখে। **Self-hosted runner offline:** `./run.sh` চলছে কি না, নেটওয়ার্ক ও token-এর মেয়াদ দেখুন। **Docker পাওয়া যাচ্ছে না:** Runner-এর OS অনুযায়ী Docker প্রি-ইনস্টল থাকে কি না দেখুন — `ubuntu-latest`-এ থাকে, সব image-এ নয়।

Cleanup

Self-hosted runner পরীক্ষা শেষে GitHub-এ Settings → Actions → Runners থেকে সেটি Remove করুন, এবং মেশিন থেকে runner ফোল্ডার মুছুন। `inspect-runner.yml` রেখে দিতে পারেন।

Reflection প্রশ্ন

১. Hosted runner প্রতিবার নতুন VM হওয়ার সুবিধা ও অসুবিধা কী? ২. Self-hosted runner কখন সস্তা, আর কখন নিরাপত্তা ঝুঁকি? ৩. Runner-এর log-এ secret ছাপা হলে কী করবেন?

ল্যাব 7

Reusable Workflow তৈরি করা

উদ্দেশ্য (Objective): একই CI ধাপ বারবার না লিখে একটি reusable workflow বানানো — যাকে অন্য workflow `workflow_call` দিয়ে ডাকে। অনেক রিপোজিটরি বা অনেক workflow থাকলে এটাই DRY (Don't Repeat Yourself) নীতি রক্ষা করে।

পূর্বশর্ত (Prerequisites): ল্যাব ২ ও ৫ সম্পূর্ণ। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:

Caller workflow (ci.yml) ── workflow_call ──▶ Reusable workflow (reusable-test.yml) → Job: test (checkout → setup-node → npm ci → npm test) → ফলাফল caller-এ ফেরে

একটি জায়গায় বদল → সব caller-এ একই আচরণ

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/reusable-test.yml
name: Reusable Test
on:
  workflow_call:
    inputs:
      node-version:
        required: false
        type: string
        default: '20'
permissions: { contents: read }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: ${{ inputs.node-version }}, cache: npm }
      - run: npm ci
      - run: npm test

# .github/workflows/ci.yml — caller
name: CI
on: { push: { branches: [main] }, pull_request: { branches: [main] } }
permissions: { contents: read }
jobs:
  call-test:
    uses: ./.github/workflows/reusable-test.yml
    with: { node-version: '20' }
```

Field ব্যাখ্যা: `on.workflow_call` — এই workflow সরাসরি trigger হয় না, শুধু ডাকলে চলে; `inputs` — caller থেকে পাওয়া মান; `uses: ./.github/...` — একই রিপোজিটরির workflow ডাকা; অন্য রিপোজিটরির workflow ডাকতে `owner/repo/.github/workflows/file.yml@v1` লিখতে হয়।

ধাপে ধাপে নির্দেশনা

ধাপ ১: `reusable-test.yml` তৈরি করুন। **ধাপ ২:** `ci.yml`-কে caller হিসেবে বদলে ফেলুন। **ধাপ ৩:** Push করে দেখুন — Actions ট্যাবে caller run-এর ভেতরে reusable job চলে। **ধাপ ৪:** Node সংস্করণ `'22'` দিয়ে ডেকে দেখুন input কাজ করে কি না (ল্যাব অ্যাপ Node 20-এ লেখা হলে 22-এও চলা উচিত; না চললে সেটাই শিক্ষা — সংস্করণ বদলের প্রভাব)। **ধাপ ৫:** Reusable workflow-এ lint step যোগ করুন — দেখুন caller-এর কোনো পরিবর্তন ছাড়াই আচরণ বদলে যায়।

যাচাই ও প্রত্যাশিত ফলাফল

Caller run-এ test job সবুজ; reusable workflow-এর নাম run-এর ভেতরে দেখা যায়; input বদলালে Node সংস্করণের log বদলায়; reusable-এ যোগ করা lint step caller-এও চলে।

সাধারণ ব্যর্থতা ও সমাধান

Reusable workflow সরাসরি চালাতে চাইলে: `workflow_call`-শুধু workflow Actions ট্যাবে “Run workflow” বোতাম দেখায় না — এটাই স্বাভাবিক। **`uses` পথ ভুল:** একই রিপোজিটরিতে `./.github/workflows/...` লিখতে হয়, শুধু ফাইলের নাম নয়। **Secret পাস হচ্ছে না:** Secret `inputs` দিয়ে নয়, `secrets: inherit` বা নাম ধরে পাস করতে হয় — ল্যাব ১২-এ বিস্তারিত।

Cleanup

পরীক্ষার জন্য বদলানো Node সংস্করণ `'20'`-এ ফিরিয়ে আনুন। দুটি workflow ফাইলই রেখে দিন — ল্যাব ৮-এর composite action-এর সঙ্গে তুলনা করতে লাগবে।

Reflection প্রশ্ন

১. Reusable workflow আর কপি-পেস্ট করা workflow-এর রক্ষণাবেক্ষণ খরচের পার্থক্য কী? ২. একই রিপোজিটরি বনাম অন্য রিপোজিটরির reusable workflow — কখন কোনটি? ৩. Reusable workflow-এ ভুল করলে সব caller-এ একসাথে ভাঙে — এই ঝুঁকি কীভাবে সামলাবেন (version pin)?

ল্যাব ৪

একটি Composite Action তৈরি করা

উদ্দেশ্য (Objective): বারবার ব্যবহৃত কয়েকটি step-কে একটি composite action-এ বাঁধা — যেমন "Node সেটআপ + install + lint" — যাতে প্রতিটি job-এ এক লাইনে ডাকা যায়। Reusable workflow পুরো job পুনঃব্যবহার করে; composite action কয়েকটি step পুনঃব্যবহার করে — পার্থক্যটি হাতে-কলমে বুঝুন।

পূর্বশর্ত (Prerequisites): ল্যাব ৭ সম্পূর্ণ। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
Job: test → Step: uses: ./node-setup (composite action) → ভেতরে: setup-node → npm ci → npm run lint → তারপর Step: npm test (job-এর নিজের step)
Composite action = step-এর প্যাকেজ; Reusable workflow = job-এর প্যাকেজ

সম্পূর্ণ কনফিগারেশন

```
# .github/actions/node-setup/action.yml
name: 'Node Setup'
description: 'Setup Node.js, install deps and lint (TechNova lab)'
inputs:
  node-version:
    description: 'Node.js version'
    required: false
    default: '20'
runs:
  using: 'composite'
  steps:
    - uses: actions/setup-node@v4
      with: { node-version: ${{ inputs.node-version }}, cache: npm }
    - run: npm ci
      shell: bash
    - run: npm run lint
      shell: bash

# ব্যবহার — .github/workflows/ci.yml-এর কোনো job-এ:
#   steps:
#     - uses: actions/checkout@v4
#     - uses: ./.github/actions/node-setup
#       with: { node-version: '20' }
#     - run: npm test
```

Field ব্যাখ্যা: `runs.using: composite` — এটি composite action; প্রতিটি `run` step-এ `shell: bash` বাধ্যতামূলক — না দিলে action ব্যর্থ হয়; `inputs` দিয়ে মান পাস করা যায়।

ধাপে ধাপে নির্দেশনা

ধাপ ১: `.github/actions/node-setup/action.yml` তৈরি করুন। **ধাপ ২:** `ci.yml`-এর test job-এ checkout-এর পর composite action ডাকুন, তারপর `npm test` step রাখুন। **ধাপ ৩:** Push করে log-এ composite action-এর

ভেতরের তিনটি step আলাদা করে চলা দেখুন। **ধাপ ৪:** Action-এ একটি ভুল করুন (যেমন lint-এর নাম ভুল) — দেখুন কোন job-গুলো ভাঙে। **ধাপ ৫:** ঠিক করুন; তারপর একটি দ্বিতীয় job-এও একই action ব্যবহার করে দেখুন — একই কোড, দুই জায়গায়।

যাচাই ও প্রত্যাশিত ফলাফল

দুটি job-ই একই composite action ব্যবহার করে সবুজ হয়; log-এ “Node Setup” step-এর ভেতরে setup-node, install ও lint দেখা যায়; action-এ বদল করলে দুটি job-এই প্রতিফলিত হয়।

সাধারণ ব্যর্থতা ও সমাধান

`shell` নেই বলে ব্যর্থ: Composite action-এর প্রতিটি `run` step-এ `shell: bash` যোগ করুন। **Action পাওয়া যাচ্ছে না:** পথ `./.github/actions/node-setup` ঠিক কি না, `action.yml` নাম ঠিক কি না দেখুন। **Cache কাজ করছে না:** `setup-node`-এর `cache: npm` composite-এর ভেতরেও কাজ করে — তবে lockfile-এর পথ ডিফল্টের বাইরে হলে `cache-dependency-path` লাগে।

Cleanup

পরীক্ষার ভুলটি ঠিক করে রাখুন। Composite action ও caller workflow — দুটোই রেখে দিন; ল্যাব ২০-এ এটিতে test step যোগ হবে।

Reflection প্রশ্ন

১. Composite action কখন, আর reusable workflow কখন — একটি সিদ্ধান্ত-নিয়ম লিখুন। ২. Composite action-এ `shell` বাধ্যতামূলক কেন? ৩. Marketplace-এর third-party action ব্যবহারের আগে কী যাচাই করবেন (ল্যাব ২২-এর সঙ্গে মিলিয়ে)?

ল্যাব 9

Matrix Testing কনফিগার করা

উদ্দেশ্য (Objective): একই test একাধিক Node.js সংস্করণ ও OS-এ সমান্তরালে চালানো — matrix strategy দিয়ে। এতে জানা যায় অ্যাপটি শুধু আপনার মেশিনে নয়, সব সমর্থিত পরিবেশে চলে কি না।

পূর্বশর্ত (Prerequisites): ল্যাব ২ ও ৫ সম্পূর্ণ। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
Matrix: node-version [18, 20, 22] × os [ubuntu-latest] → ৩টি সমান্তরাল job: (18, ubuntu) (20, ubuntu) (22, ubuntu) → প্রতিটির ফল আলাদা; একটি লাল হলে বোঝা যায় কোন সংস্করণে ভাঙে

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/matrix.yml
name: Matrix Test
on: { push: { branches: [main] }, pull_request: { branches: [main] } }
permissions: { contents: read }
jobs:
  test:
    strategy:
      fail-fast: false
      matrix:
        node-version: ['18', '20', '22']
        os: [ubuntu-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: ${{ matrix.node-version }}, cache: npm }
      - run: npm ci
      - run: npm test
```

Field ব্যাখ্যা: `matrix` — প্রতিটি মানের combination-এ একটি করে job তৈরি হয়; `fail-fast: false` — একটি job ব্যর্থ হলেও বাকিগুলো চলতে থাকে (কোন সংস্করণে ভাঙে, তা জানতে দরকার); `${{ matrix.node-version }}` — বর্তমান job-এর মান।

ধাপে ধাপে নির্দেশনা

ধাপ ১: `matrix.yml` তৈরি করে push করুন। **ধাপ ২:** Actions ট্যাবে ৩টি job সমান্তরালে চলা দেখুন। **ধাপ ৩:** একটি Node 18-এ ভাঙে এমন কোড যোগ করুন (যেমন Node 20-এর নতুন API ব্যবহার) — দেখুন শুধু 18-এর job লাল হয়, বাকি দুটি সবুজ। **ধাপ ৪:** `fail-fast: true` করে আবার চালান — পার্থক্য লক্ষ করুন। **ধাপ ৫:** `fail-fast: false`-এ ফিরিয়ে আনুন ও ভাঙা কোডটি সরান।

যাচাই ও প্রত্যাশিত ফলাফল

৩টি job-এর ফল আলাদা আলাদা দেখা যায়; ভাঙা সংস্করণ শনাক্ত করা যায়; `fail-fast: false`-এ সব job-এর ফল পাওয়া যায়।

সাধারণ ব্যর্থতা ও সমাধান

Job সংখ্যা প্রত্যাশার চেয়ে বেশি: Matrix-এর প্রতিটি dimension গুণ হয় — ৩ সংস্করণ × ২ OS = ৬ job; dimension যোগ করার আগে গুণফল হিসাব করুন। **সব job একই সংস্করণে চলছে:** `setup-node`-এ `${{ matrix.node-version }}` লেখা হয়েছে কি না দেখুন — হার্ডকোড করা '20' থাকলে matrix অর্থহীন। **খরচ/সময় বেশি:** PR-এ ছোট matrix, main-এ বড় matrix চালান — ল্যাব ১১-এর optimization-এর সঙ্গে মিলিয়ে ভাবুন।

Cleanup

পরীক্ষার ভাঙা কোডটি সরিয়ে ফেলুন। `matrix.yml` রেখে দিন — প্রজেক্ট ১-এ এটি CI-এর অংশ হবে।

Reflection প্রশ্ন

১. Matrix কখন উপকারী, আর কখন শুধু খরচ বাড়ায়? ২. `fail-fast: false` কখন ভালো, কখন খারাপ? ৩. OS matrix (ubuntu/windows/macos) কখন দরকার, MERN অ্যাপে কখন নয়?

ল্যাব 10

Workflow Caching বাস্তবায়ন করা

উদ্দেশ্য (Objective): `node_modules` ও npm cache ধরে রেখে প্রতিটি run-এর install সময় কমানো। Cache ঠিক থাকলে একই dependency বারবার ডাউনলোড হয় না; cache key ভুল হলে পুরনো dependency দিয়ে ভুল ফল আসতে পারে — দুটোই মেপে দেখুন।

পূর্বশর্ত (Prerequisites): ল্যাব ২ সম্পূর্ণ; workflow-এ `npm ci` আছে। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
Run 1: npm ci (ডাউনলোড) → Cache save (key: package-lock.json-এর hash) → Run 2: Cache hit → install প্রায় সঙ্গে সঙ্গে → Run 3 (lockfile বদলেছে): Cache miss → নতুন ডাউনলোড → নতুন cache

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/ci.yml-এর step (দুটি পদ্ধতি — একটি বেছে নিন)
# পদ্ধতি A — setup-node-এর বিল্ট-ইন cache (সহজ, সুপারিশকৃত):
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
# পদ্ধতি B — স্পষ্ট cache step (শেখার জন্য):
      - uses: actions/cache@v4
        with:
          path: ~/.npm
          key: npm-${{ runner.os }}-${{ hashFiles('**/package-lock.json') }}
          restore-keys: |
            npm-${{ runner.os }}-
      - run: npm ci
```

Field ব্যাখ্যা: `key` — lockfile-এর hash দিয়ে তৈরি; lockfile বদলালে key বদলায়, পুরনো cache ব্যবহৃত হয় না; `restore-keys` — ঠিক key না মিললে কাছাকাছি পুরনো cache ব্যবহার করে (আংশিক hit)।

ধাপে ধাপে নির্দেশনা

ধাপ ১: Cache ছাড়া একটি run-এর সময় লিখুন (Actions ট্যাবে প্রতিটি step-এর সময় দেখা যায়)। **ধাপ ২:** পদ্ধতি A চালু করে দুবার push করুন — দ্বিতীয় run-এ “Cache restored” ও কম সময় দেখুন। **ধাপ ৩:** একটি dependency যোগ করুন (`npm install lodash` — শুধু ল্যাবের জন্য), push করুন — cache miss ও নতুন install দেখুন। **ধাপ ৪:** আবার push করুন (কোডে ছোট পরিবর্তন) — নতুন lockfile-এর cache hit দেখুন। **ধাপ ৫:** দুই পদ্ধতির সময় একটি টেবিলে লিখুন।

যাচাই ও প্রত্যাশিত ফলাফল

দ্বিতীয় run-এ install step-এর সময় প্রথম run-এর চেয়ে স্পষ্ট কম; log-এ “Cache restored from key ...” দেখা যায়; lockfile বদলের run-এ cache miss হয়, তার পরের run-এ আবার hit হয়।

সাধারণ ব্যর্থতা ও সমাধান

Cache hit হচ্ছে না: Key-তে `hashFiles('**/package-lock.json')` ঠিক আছে কি না, lockfile commit করা আছে কি না দেখুন। **পুরনো dependency চলছে:** Key-তে lockfile hash না থাকলে পুরনো cache বারবার ব্যবহৃত হয় — key-তে hash যোগ করুন। **Cache সাইজ সীমা:** GitHub cache-এর সীমা আছে — বড় `node_modules` সরাসরি cache করবেন না, `~/.npm` বা setup-node cache-ই যথেষ্ট।

Cleanup

ল্যাবের জন্য যোগ করা `lodash` dependency সরান (`npm uninstall lodash`)। Cache মুছতে চাইলে GitHub-এ Actions → Caches থেকে মুছুন।

Reflection প্রশ্ন

১. Cache key-তে lockfile hash কেন অপরিহার্য? ২. Cache কখন ভুল ফল দিতে পারে, এবং কীভাবে ধরবেন? ৩. `npm ci`-এর সঙ্গে cache ব্যবহার করলে কী সুবিধা থাকে, `npm install`-এর সঙ্গে কী ঝুঁকি?

ল্যাব 11

Artifact Upload ও Download করা

উদ্দেশ্য (Objective): Build job-এর আউটপুট (`dist/`) artifact হিসেবে upload করা ও অন্য job-এ download করে ব্যবহার করা। Job-গুলো আলাদা runner-এ চলে — artifact-ই তাদের মধ্যে ফাইল পাঠানোর সেতু।

পূর্বশর্ত (Prerequisites): ল্যাব ৫ সম্পূর্ণ (build job আছে)। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
Job: build (npm run build → dist/) ── upload-artifact (name: app-build) ──▶ GitHub Artifact Storage ──
download-artifact ──▶ Job: verify (dist/ পড়ে, ফাইল আছে কি না যাচাই করে)

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/ci.yml — build ও verify job
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: app-build
          path: dist/
          retention-days: 7
  verify:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with: { name: app-build, path: dist/ }
      - run: ls -la dist/ && test -f dist/server.js && echo "ARTIFACT OK"
```

Field ব্যাখ্যা: `name` — artifact-এর নাম, download-এ একই নাম লাগে; `path` — কোন ফোল্ডার/ফাইল upload হবে; `retention-days: 7` — ৭ দিন পর artifact মুছে যাবে (খরচ ও গোপনীয়তার জন্য গুরুত্বপূর্ণ)।

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের দুটি job লিখে push করুন। **ধাপ ২:** Run শেষে GitHub-এ run পাতার নিচে Artifacts অংশে `app-build` দেখুন ও ডাউনলোড করে ভেতরের ফাইল মিলিয়ে দেখুন। **ধাপ ৩:** Verify job-এর log-এ “ARTIFACT OK” দেখুন। **ধাপ ৪:** ইচ্ছা করে build-এ `dist/` তৈরি না হওয়ার মতো ভুল করুন — upload step ব্যর্থ হওয়া দেখুন। **ধাপ ৫:** ঠিক করে আবার চালান।

যাচাই ও প্রত্যাশিত ফলাফল

Run পাতায় artifact দেখা যায় ও ডাউনলোড করা যায়; verify job সবুজ ও “ARTIFACT OK” ছাপে; ভুল build-এ upload ব্যর্থ হয় — ফলে ভাঙা artifact কখনো পরের ধাপে যায় না।

সাধারণ ব্যর্থতা ও সমাধান

`path` **পাওয়া যাচ্ছে না:** Build step-এ `dist/` আসলেই তৈরি হয়েছে কি না, `.gitignore` artifact-কে আটকাচ্ছে কি না দেখুন (artifact upload gitignore মানে না, তবে build script-এর ভুল পথ দেখুন)। **নাম মিলছে না:** Upload ও download-এ `name` হুবহু এক হতে হবে। **Artifact অনেক বড়:** `node_modules` upload করবেন না — শুধু build আউটপুট upload করুন।

Cleanup

Run পাতা থেকে পুরনো artifact মুছে ফেলুন (Artifacts → Delete)। `retention-days` কম রাখুন যাতে artifact জমে না থাকে।

Reflection প্রশ্ন

১. Artifact আর cache-এর পার্থক্য কী — কখন কোনটি? ২. Artifact-এ secret বা `.env` ফাইল গেলে কী ঝুঁকি? ৩. Retention period কম রাখা কেন খরচ ও নিরাপত্তা — দুটোর জন্যই ভালো?

ল্যাব 12

Secret ও Environment কনফিগার করা

উদ্দেশ্য (Objective): GitHub Secrets ও Environments ব্যবহার করে pipeline-এ নিরাপদে মান পাঠানো — কোডে secret না লিখে, staging ও production-এর মান আলাদা রেখে, production deploy-এ অনুমোদন (approval) বসিয়ে।

পূর্বশর্ত (Prerequisites): ল্যাব ২ সম্পূর্ণ। **প্রয়োজনীয় টুল:** GitHub, Git, Node.js 20।

আর্কিটেকচার:
GitHub Secrets (repo / environment) → Workflow: ${{ secrets.CHANGE_ME_SECRET }} → Job (environment: staging) → Deploy step
Environment: production → Required reviewers: 1 জন → অনুমোদন ছাড়া deploy job শুরু হয় না

সতর্কতা: এই ল্যাবে শুধু কাল্পনিক মান ব্যবহার করুন — যেমন `CHANGE-ME-SECRET`। আসল API key বা token কখনো Secret-এও টেস্টের জন্য দেবেন না, যদি না সেটি revoke করা যায় এমন টেস্ট key হয়। Secret-এর মান কখনো log-এ ছাপাবেন না — GitHub `***` দিয়ে ঢাকে, তবুও ইচ্ছা করে ছাপাবেন না।

সম্পূর্ণ কনফিগারেশন

```
# GitHub UI: Settings → Secrets and variables → Actions → New repository secret
#   Name: TECHNOVA_API_KEY   Value: CHANGE-ME-SECRET
# Settings → Environments → New environment: staging → (production-এ Required reviewers যোগ করুন)
# .github/workflows/deploy.yml
name: Deploy
on: { workflow_dispatch: {} }
permissions: { contents: read }
jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - run: echo "Deploying to staging with key length check..."
      - run: |
          if [ -z "${{ secrets.TECHNOVA_API_KEY }}" ]; then echo "SECRET MISSING"; exit 1; fi
          echo "Secret is set (value never printed)"
  deploy-prod:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: echo "Production deploy — approved only"
```

Field ব্যাখ্যা: `environment: staging` — job-টি ওই environment-এর secret/variable পায়; `environment: production` + Required reviewers — অনুমোদন না পেলে job অপেক্ষায় থাকে; `secrets.TECHNOVA_API_KEY` — মানটি log-এ কখনো দেখা যায় না।

ধাপে ধাপে নির্দেশনা

ধাপ ১: Repository secret `TECHNOVA_API_KEY = CHANGE-ME-SECRET` তৈরি করুন। **ধাপ ২:** `staging` ও `production` environment তৈরি করুন; production-এ নিজেকে Required reviewer হিসেবে যোগ করুন। **ধাপ ৩:** উপরের workflow লিখে `workflow_dispatch` দিয়ে চালান। **ধাপ ৪:** Staging job সবুজ হওয়া ও production job “Waiting for approval” অবস্থায় থাকা দেখুন — অনুমোদন দিন, তারপর সেটি চলা দেখুন। **ধাপ ৫:** Secret মুছে আবার চালান — “SECRET MISSING” দিয়ে job ব্যর্থ হওয়া দেখুন, তারপর secret ফিরিয়ে আনুন।

যাচাই ও প্রত্যাশিত ফলাফল

Secret থাকলে staging job সবুজ; secret না থাকলে job ব্যর্থ; production job অনুমোদন ছাড়া চলে না; log-এ secret-এর মান কোথাও দেখা যায় না।

সাধারণ ব্যর্থতা ও সমাধান

Secret খালি দেখাচ্ছে: Secret-এর নাম YAML-এর সঙ্গে হুবহু মিলছে কি না দেখুন (বড় হাতের অক্ষরসহ)। **Approval আসছে না:** Environment-এর Required reviewers-এ সঠিক ব্যক্তি আছেন কি না, এবং তিনি নিজের run অনুমোদন করতে পারছেন কি না দেখুন (GitHub-এ নিজের run নিজে অনুমোদন করা যায় না — ল্যাবে অন্য অ্যাকাউন্ট বা reviewer নিয়ম শিথিল করে পরীক্ষা করুন)। **Environment secret পাওয়া যাচ্ছে না:** Secret-টি repository-তে নয়, environment-এ তৈরি হয়েছে কি না দেখুন।

Cleanup

টেস্ট secret-টি মুছে ফেলুন বা কাল্পনিক মানে রেখে দিন। Production environment-এর reviewer নিয়ম ল্যাব রিপোজিটরিতে রেখে দিতে পারেন।

Reflection প্রশ্ন

১. Repository secret, environment secret ও variable — কখন কোনটি? ২. Secret log-এ ছাপা হলে কী করবেন (revoke, rotate)? ৩. Production-এ অনুমোদন না থাকলে কী কী দুর্ঘটনা ঘটতে পারে?

ল্যাব 13

OIDC Authentication নিরাপদে প্রদর্শন করা

উদ্দেশ্য (Objective): দীর্ঘমেয়াদি (long-lived) credential সংরক্ষণ না করে, GitHub-এর OIDC token দিয়ে স্বল্পমেয়াদি পরিচয় প্রমাণের ধারণাটি নিরাপদে প্রদর্শন করা — কোনো আসল cloud অ্যাকাউন্টে সংযোগ না করে। এতে বোঝা যায় কেন আধুনিক pipeline-এ static key রাখা আর দরকার হয় না।

পূর্বশর্ত (Prerequisites): ল্যাব ১২ সম্পূর্ণ; OIDC ধারণা (অধ্যায় ১০) পড়া আছে। **প্রয়োজনীয় টুল:** GitHub, Git; কোনো আসল cloud credential লাগবে না।

আর্কিটেকচার:

GitHub Actions → OIDC token (স্বল্পমেয়াদি, এই run-এর জন্য) → Cloud Identity Provider (AWS/Azure/GCP — ল্যাবে শুধু ধারণা) → যাচাই: token-এর দাবি (repo, branch, environment) → স্বল্পমেয়াদি cloud credential → Deploy

ল্যাবে: token-এর অস্তিত্ব ও দাবি যাচাই করা হবে, আসল cloud-এ যাওয়া হবে না

সতর্কতা: এই ল্যাবের উদ্দেশ্য শেখা, আসল cloud সংযোগ নয়। কোনো আসল AWS/Azure/GCP অ্যাকাউন্টে OIDC trust বসাবেন না যতক্ষণ না আপনি trust policy-র প্রতিটি শর্ত বুঝেছেন। ভুল trust policy (যেমন যেকোনো branch-কে অনুমতি) প্রোডাকশন অ্যাকাউন্ট খুলে দিতে পারে।

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/oidc-demo.yml
name: OIDC Demo
on: { workflow_dispatch: {} }
permissions:
  contents: read
  id-token: write   # OIDC token চাইতে এই permission লাগে — শুধু এই workflow-এ
jobs:
  demo:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - name: Request OIDC token (demo only — no cloud call)
        run: |
          echo "Requesting token from $ACTIONS_ID_TOKEN_REQUEST_URL"
          # টোকেনের মান কখনো ছাপাবেন না — শুধু অস্তিত্ব ও দাবি যাচাই করুন
          TOKEN=$(curl -s -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
            "$ACTIONS_ID_TOKEN_REQUEST_URL?audience=api://AzureADTokenExchange" | python3 -c
"import sys,json; print(json.load(sys.stdin).get('value',''))")
          if [ -z "$TOKEN" ]; then echo "NO TOKEN"; exit 1; fi
          echo "Token received (length: ${#TOKEN} chars, value NOT printed)"
          echo "$TOKEN" | cut -d. -f2 | tr '_-' '/+' | python3 -c "
import sys,base64,json
s=sys.stdin.read().strip(); s+='='*(-len(s)%4)
print(json.dumps(json.loads(base64.b64decode(s)), indent=2))" | head -20
          # নমুনা আউটপুট — চিত্রণমূলক
```

```
          # { "sub": "repo:OWNER/technova:environment:staging", "aud": "...", "iss": "https://
token.actions.githubusercontent.com" }
```

Field ব্যাখ্যা: `id-token: write` — workflow-কে OIDC token চাওয়ার অনুমতি দেয়; `sub` দাবি — কোন repo/branch/environment থেকে token এসেছে, তা বলে; cloud-এ trust policy এই `sub`-কে মিলিয়ে অনুমতি দেয়।

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের workflow লিখে `workflow_dispatch` দিয়ে চালান। **ধাপ ২:** Log-এ token-এর দাবিগুলো (claims) দেখুন — `sub`, `aud`, `iss`, `repository` মিলিয়ে দেখুন। **ধাপ ৩:** Environment বদলে (staging → production) আবার চালান — `sub`-এর মান বদলানো দেখুন। **ধাপ ৪:** `id-token: write` সরিয়ে আবার চালান — token না পাওয়া দেখুন। **ধাপ ৫:** লিখুন: static AWS key সংরক্ষণ করা বনাম OIDC — কোনটিতে key চুরি হলে ক্ষতি বেশি, এবং কেন।

যাচাই ও প্রত্যাশিত ফলাফল

Token পাওয়া যায়, তার দাবিতে সঠিক repo ও environment দেখা যায়, token-এর মান log-এ কোথাও ছাপা হয় না; `id-token: write` না থাকলে token পাওয়া যায় না।

সাধারণ ব্যর্থতা ও সমাধান

Token পাওয়া যাচ্ছে না: `permissions` workflow-স্তরে না job-স্তরে — দুটো জায়গাতেই দেওয়া যায়, তবে job-স্তরে দিলে স্পষ্ট হয়। **Decode ব্যর্থ:** Base64 padding-এর জন্য `tr` ও padding লাইনটি ঠিক আছে কি না দেখুন — এটি শেখার জন্য, প্রোডাকশনে token decode করে যাচাই করবেন না। **Audience ভুল:** Audience cloud provider অনুযায়ী বদলায় (AWS: `sts.amazonaws.com`) — ল্যাবে audience শুধু প্রদর্শনের জন্য।

Cleanup

OIDC token স্বল্পমেয়াদি — run শেষে নিজেই মেয়াদোত্তীর্ণ হয়, মুছতে হয় না। Workflow ফাইলটি রেখে দিন; আসল cloud trust কনফিগারেশন এই ল্যাবে করবেন না।

Reflection প্রশ্ন

১. Static credential চুরি হলে আর OIDC token চুরি হলে — ক্ষতির পার্থক্য কী? ২. Trust policy-তে `sub` শর্ত না থাকলে কী ঝুঁকি? ৩. কোন environment-কে OIDC দিয়ে production deploy-এর অনুমতি দেওয়া উচিত, আর কাকে নয়?

ল্যাব 14

একটি GitLab CI Pipeline তৈরি করা

উদ্দেশ্য (Objective): একই Node.js অ্যাপের জন্য GitLab CI/CD pipeline লেখা — `.gitlab-ci.yml`-এ stages, jobs ও rules ব্যবহার করে। GitHub Actions জানা থাকলে GitLab-এর ধারণা মেলানো সহজ হয় — পার্থক্যগুলো নিজে দেখুন।

পূর্বশর্ত (Prerequisites): GitLab অ্যাকাউন্ট (gitlab.com) ও একটি টেস্ট প্রজেক্ট; Node.js অ্যাপ। **প্রয়োজনীয় টুল:** GitLab, Git, Node.js 20।

আর্কিটেকচার:

git push → GitLab → .gitlab-ci.yml → Pipeline: Stage test (lint + unit test) → Stage build (npm run build) → Stage deploy (manual, শুধু main) → Runner (shared runner) প্রতিটি job চালায়

সম্পূর্ণ কনফিগারেশন

```
# .gitlab-ci.yml
stages: [test, build, deploy]
image: node:20
variables:
  NODE_ENV: test
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths: [.npm/]
  policy: pull-push
before_script:
  - npm ci --cache .npm --prefer-offline
lint:
  stage: test
  script: [npm run lint]
unit-test:
  stage: test
  script: [npm test]
build:
  stage: build
  script: [npm run build]
  artifacts:
    paths: [dist/]
    expire_in: 1 week
deploy-staging:
  stage: deploy
  script: [echo "Deploying to staging.technova.example"]
  environment: { name: staging, url: https://staging.technova.example }
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Field ব্যাখ্যা: `stages` — ধাপের ক্রম; `image: node:20` — job-গুলো এই Docker image-এ চলে; `cache` — dependency cache; `artifacts` — build আউটপুট সংরক্ষণ; `rules ... when: manual` — deploy শুধু main-এ, হাতে চাপ দিলে চলে।

ধাপে ধাপে নির্দেশনা

ধাপ ১: GitLab-এ টেস্ট প্রজেক্ট তৈরি করে একই Node অ্যাপ push করুন। **ধাপ ২:** `.gitlab-ci.yml` যোগ করে push করুন। **ধাপ ৩:** CI/CD → Pipelines-এ pipeline-এর stage-গুলো চলা দেখুন। **ধাপ ৪:** Deploy job-এ "Play" বোতাম চেপে manual deploy চালান। **ধাপ ৫:** Test ভেঙে push করুন — build ও deploy stage না চলা দেখুন। **ধাপ ৬:** ঠিক করে আবার চালান।

যাচাই ও প্রত্যাশিত ফলাফল

Pipeline-এ test → build → deploy ক্রমে চলে; deploy job manual (▶ বোতাম) অবস্থায় থাকে; artifact ডাউনলোড করা যায়; ভাঙা test-এ পরের stage চলে না।

সাধারণ ব্যর্থতা ও সমাধান

Pipeline তৈরি হচ্ছে না: YAML indentation ও `stages`-এর নাম মিলছে কি না দেখুন; GitLab-এ CI/CD → Editor-এ "Validate" দিয়ে যাচাই করুন। **Runner নেই (stuck):** Shared runner চালু আছে কি না দেখুন (Settings → CI/CD → Runners)। **`npm ci` ব্যর্থ:** `package-lock.json` commit করা আছে কি না দেখুন।

Cleanup

টেস্ট প্রজেক্টটি GitLab-এ রেখে দিন (ল্যাব ১৫–১৬-এ লাগবে) অথবা মুছে ফেলুন — তবে ল্যাব ১৫-এর আগে মুছবেন না।

Reflection প্রশ্ন

১. GitLab-এর stage আর GitHub Actions-এর job-এর পার্থক্য কী? ২. `when: manual` কখন ভালো, কখন বিরক্তিকর? ৩. একই অ্যাপের GitHub ও GitLab pipeline — কোনটি পড়তে সহজ লাগল, এবং কেন?

ল্যাব 15

একটি GitLab Runner কনফিগার করা

উদ্দেশ্য (Objective): নিজের মেশিনে GitLab Runner ইনস্টল করে প্রজেক্টে নিবন্ধন (register) করা ও Docker executor দিয়ে job চালানো। Shared runner-এর সীমা বুঝলে নিজের runner কখন দরকার, তা স্পষ্ট হয়।

পূর্বশর্ত (Prerequisites): ল্যাব ১৪ সম্পূর্ণ; Docker Engine চালু আছে এমন একটি ল্যাব VM বা কম্পিউটার; GitLab প্রজেক্টে Maintainer অধিকার। **প্রয়োজনীয় টুল:** GitLab, Docker, GitLab Runner binary।

আর্কিটেকচার:
GitLab → Runner Manager (আপনার মেশিন) → Executor: docker → প্রতিটি job-এর জন্য নতুন container (node:20) → Job শেষে container মুছে যায়
ট্যাগ (tags): docker-lab — শুধু এই ট্যাগের job-গুলো এই runner-এ চলে

সতর্কতা: Runner-টি শুধু টেস্ট প্রজেক্টে ব্যবহার করুন। Shell executor-এ অচেনা কোড চালাবেন না — Docker executor অনেক বেশি বিচ্ছিন্ন (isolated)। Runner-এর registration token কখনো কোডে বা স্ক্রিনশটে দেবেন না।

সম্পূর্ণ কনফিগারেশন

```
# Linux-এ Runner ইনস্টল (GitLab-এর অফিসিয়াল নির্দেশনা অনুযায়ী — সংস্করণ যাচাই করে নিন):
#   curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/
script.deb.sh" | sudo bash
#   sudo apt install gitlab-runner
# নিবন্ধন (GitLab UI: Settings → CI/CD → Runners → New project runner থেকে token নিন):
sudo gitlab-runner register \
  --url https://gitlab.com \
  --token CHANGE-ME-TOKEN \
  --executor docker \
  --docker-image node:20 \
  --description "lab-docker-runner" \
  --tag-list "docker-lab" \
  --run-untagged="false"
# .gitlab-ci.yml-এ ট্যাগ ব্যবহার:
# lint:
#   tags: [docker-lab]
#   stage: test
#   script: [npm run lint]
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: GitLab-এ প্রজেক্টের Runners পাতা থেকে registration token নিন (token-টি শুধু একবার দেখা যায় — কোথাও সংরক্ষণ করবেন না)। **ধাপ ২:** Runner ইনস্টল ও register করুন — executor `docker`, image `node:20`, tag `docker-lab`। **ধাপ ৩:** `sudo gitlab-runner verify` ও `sudo gitlab-runner status` দিয়ে runner চালু আছে কি না দেখুন। **ধাপ ৪:** একটি job-এ `tags: [docker-lab]` যোগ করে push করুন — job আপনার runner-এ চলা দেখুন (job log-এ runner-এর নাম দেখা যায়)। **ধাপ ৫:** Runner বন্ধ করে (`sudo gitlab-runner stop`) আবার push করুন — job আটকে (stuck) থাকা দেখুন, তারপর runner চালু করে job চলা দেখুন।

যাচাই ও প্রত্যাশিত ফলাফল

GitLab-এ Runners পাতায় runner “online” সবুজ; ট্যাগ করা job আপনার runner-এ চলে; runner বন্ধ থাকলে job pending থাকে; চালু করলে job চলে।

সাধারণ ব্যর্থতা ও সমাধান

Runner offline: `gitlab-runner status` ও `config.toml`-এ URL/token ঠিক আছে কি না দেখুন। **Job stuck:** Job-এর tag আর runner-এর tag মিলছে কি না, এবং `run-untagged` সেটিং দেখুন। **Docker permission:** `gitlab-runner` ইউজার Docker group-এ আছে কি না দেখুন (`sudo usermod -aG docker gitlab-runner`)।

Cleanup

পরীক্ষা শেষে GitLab-এ runner-টি Remove করুন ও মেশিন থেকে `sudo gitlab-runner unregister --all-runners` চালান। Runner uninstall করতে চাইলে `sudo apt remove gitlab-runner`।

Reflection প্রশ্ন

১. Docker executor আর shell executor-এর নিরাপত্তা পার্থক্য কী? ২. Tag দিয়ে job রাউট করা কেন দরকার? ৩. Shared runner বনাম নিজের runner — খরচ, গতি ও রক্ষণাবেক্ষণের তুলনা লিখুন।

ল্যাব 16

GitLab Artifact ও Cache নিয়ে কাজ করা

উদ্দেশ্য (Objective): GitLab-এ artifact (job-এর মধ্যে ফাইল পাঠানো) ও cache (dependency পুনঃব্যবহার) — দুটোর পার্থক্য হাতে-কলমে বোঝা। Artifact পরের stage-এ যায়; cache একই job-এর পরের run-এ সময় বাঁচায়।

পূর্বশর্ত (Prerequisites): ল্যাব ১৪ সম্পূর্ণ। **প্রয়োজনীয় টুল:** GitLab, Git, Node.js 20।

আর্কিটেকচার:
Job: build → artifacts: dist/ (expire_in: 1 week) ⟶ Job: test-dist (dependencies: [build] — dist/ পায়)
Cache: .npm/ → পরের pipeline run-এ npm ci দ্রুত হয় (key: CI_COMMIT_REF_SLUG — branch অনুযায়ী আলাদা cache)

সম্পূর্ণ কনফিগারেশন

```
# .gitlab-ci.yml
stages: [test, build, verify]
image: node:20
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths: [.npm/]
  policy: pull-push
before_script: [npm ci --cache .npm --prefer-offline]
unit-test:
  stage: test
  script: [npm test]
build:
  stage: build
  script: [npm run build]
  artifacts:
    paths: [dist/]
    expire_in: 1 week
verify-dist:
  stage: verify
  dependencies: [build]
  script:
    - ls -la dist/
    - test -f dist/server.js && echo "ARTIFACT OK"
```

Field ব্যাখ্যা: `dependencies: [build]` — verify job শুধু build job-এর artifact পায়, অন্য job-এর নয়; `expire_in` — artifact কতদিন থাকবে; `cache.key` — branch অনুযায়ী আলাদা cache, যাতে এক branch-এর cache অন্য branch-এ না মেশে।

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের `.gitlab-ci.yml` লিখে push করুন। **ধাপ ২:** Pipeline-এ তিনটি stage চলা দেখুন; build job-এর Artifacts থেকে `dist/` ডাউনলোড করুন। **ধাপ ৩:** দ্বিতীয়বার push করুন (ছোট পরিবর্তন) — job log-এ cache ব্যবহারের কথা ("Checking cache...") ও কম install সময় দেখুন। **ধাপ ৪:** `dependencies` লাইনটি সরিয়ে আবার চালান

— verify job-এ `dist/` না পাওয়া দেখুন (এটাই প্রমাণ যে artifact ছাড়া stage-এর মধ্যে ফাইল যায় না)। **ধাপ ৫:** `dependencies` ফিরিয়ে আনুন।

যাচাই ও প্রত্যাশিত ফলাফল

Build artifact ডাউনলোড করা যায়; verify job “ARTIFACT OK” ছাপে; দ্বিতীয় run-এ cache hit হয়; `dependencies` ছাড়া verify job ব্যর্থ হয়।

সাধারণ ব্যর্থতা ও সমাধান

Artifact খালি: `paths`-এ ফোল্ডারের নাম ঠিক কি না, build-এ `dist/` তৈরি হয়েছে কি না দেখুন। **Cache hit হচ্ছে না:** `key` স্থির কি না, `.npm` পথ `npm ci --cache .npm`-এর সঙ্গে মিলছে কি না দেখুন। **Verify job-এ পুরনো artifact:** `dependencies` ছাড়া GitLab আগের সব job-এর artifact দেয় — স্পষ্ট `dependencies` লেখাই নিরাপদ।

Cleanup

পুরনো pipeline-এর artifact GitLab-এ মুছে ফেলুন (job পাতা → Erase job artifacts)। `expire_in` ১ সপ্তাহে রাখুন।

Reflection প্রশ্ন

১. Artifact আর cache — কোনটি deployment-এর জন্য, কোনটি গতির জন্য? ২. `expire_in` না দিলে কী সমস্যা (খরচ, গোপনীয়তা)? ৩. Cache key-তে branch নাম কেন ব্যবহার করা হয়?

ল্যাব 17

Jenkins ইনস্টল ও অন্বেষণ করা

উদ্দেশ্য (Objective): একটি ল্যাব VM-এ Jenkins controller ইনস্টল করে প্রথমবার চালানো — Dashboard, plugin, job তৈরি ও build history দেখা। Jenkins নিজে হোস্ট করতে হয় — এই অপারেশনাল দায়িত্বটাই GitHub/GitLab-এর সঙ্গে এর মূল পার্থক্য।

পূর্বশর্ত (Prerequisites): Ubuntu 24.04 LTS ল্যাব VM (২ vCPU, ৪ GB RAM), Java 17, Docker ঐচ্ছিক। **প্রয়োজনীয় টুল:** Jenkins (Docker image `jenkins/jenkins:lts` সুপারিশকৃত — ল্যাবের জন্য সহজ), ব্রাউজার।

আর্কিটেকচার:
ব্রাউজার → Jenkins Controller (পোর্ট 8080) → Built-in node (প্রথম job এখানেই) → Job: technova-ci → Build history (প্রতিটি build-এর নম্বর, সময়, console output) → Workspace: /var/jenkins_home/jobs/...

সতর্কতা: Jenkins শুধু ল্যাব VM বা বিচ্ছিন্ন নেটওয়ার্কে চালান। প্রথম setup-এ admin password (`initialAdminPassword`) কাউকে দেবেন না, এবং Jenkins-কে ইন্টারনেটে খোলা পোর্টে রাখবেন না।

সম্পূর্ণ কনফিগারেশন

```
# Docker দিয়ে Jenkins চালানো (ল্যাবের জন্য সহজতম):
docker run -d --name jenkins-lab \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
docker logs jenkins-lab
# নমুনা আউটপুট — চিত্রণমূলক
# Jenkins initial setup is required. An admin user has been created...
# Password: a1b2c3d4... (আসল মান logs-এ দেখা যায় — কাউকে দেবেন না)
# অথবা ফাইল থেকে: docker exec jenkins-lab cat /var/jenkins_home/secrets/initialAdminPassword
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের Docker কমান্ডে Jenkins চালান। **ধাপ ২:** ব্রাউজারে `http://203.0.113.10:8080` খুলুন (ল্যাব VM-এর IP বসান), admin password দিয়ে Unlock করুন। **ধাপ ৩:** “Install suggested plugins” বেছে নিন, admin ইউজার তৈরি করুন (পাসওয়ার্ড: `CHANGE-ME-2026` — ল্যাবের জন্য)। **ধাপ ৪:** New Item → Freestyle project → নাম `technova-ci` → Build Steps → Execute shell → `echo "Hello Jenkins" && node --version || echo "Node not in controller"` → Save → Build Now। **ধাপ ৫:** Build History-তে build #1 → Console Output দেখুন। **ধাপ ৬:** Manage Jenkins → System Information-এ Java সংস্করণ, হোম ডিরেক্টরি ও plugin তালিকা দেখুন।

যাচাই ও প্রত্যাশিত ফলাফল

Dashboard খোলে; প্রথম build সবুজ (✅) ও console output-এ “Hello Jenkins” দেখা যায়; System Information-এ Jenkins সংস্করণ ও plugin তালিকা দেখা যায়।

সাধারণ ব্যর্থতা ও সমাধান

8080 পোর্ট খুলছে না: Container চলছে কি না (`docker ps`), ফায়ারওয়াল পোর্ট 8080 খোলা কি না দেখুন। **Password পাওয়া যাচ্ছে না:** `docker logs jenkins-lab` অথবা `secrets/initialAdminPassword` ফাইল দেখুন। **Plugin ইনস্টল ব্যর্থ:** VM-এর ইন্টারনেট সংযোগ দেখুন; পরে Manage Jenkins → Plugins থেকে আবার চেষ্টা করুন। **Node পাওয়া যাচ্ছে না:** Controller-এ Node নেই — এটাই স্বাভাবিক; Node agent-এ বা Docker agent-এ চালাতে হয় (ল্যাব ১৮–১৯)।

Cleanup

ল্যাব শেষে container থামান: `docker stop jenkins-lab`। মুছতে চাইলে `docker rm -v jenkins-lab` (volume-সহ মুছবে — ডেটা যাবে, তাই আগে backup নিন যদি job রাখতে চান)।

Reflection প্রশ্ন

১. Jenkins নিজে হোস্ট করার দায়িত্ব কী কী (আপডেট, ব্যাকআপ, নিরাপত্তা)? ২. Freestyle job আর Pipeline job-এর পার্থক্য কী? ৩. Controller-এ সরাসরি build চালানো কেন নিরাপদ নয়?

ল্যাব 18

Jenkins Agent কনফিগার করা

উদ্দেশ্য (Objective): Controller থেকে আলাদা একটি agent (node) যোগ করা — যাতে build controller-এ নয়, agent-এ চলে। এটাই Jenkins-এর controller/agent মডেলের মূল: controller পরিচালনা করে, agent কাজ করে।

পূর্বশর্ত (Prerequisites): ল্যাব ১৭ সম্পূর্ণ; Jenkins চলছে; agent হিসেবে ব্যবহারের জন্য একটি দ্বিতীয় VM অথবা Docker (Docker agent সবচেয়ে সহজ)। **প্রয়োজনীয় টুল:** Jenkins, Docker, Node.js 20 (agent image-এ)।

আর্কিটেকচার:
Jenkins Controller (8080) ── JNLP / SSH ──▶ Agent: node-agent (label: node20) → Workspace → Job চলে agent-এ → ফলাফল controller-এ ফেরে
Controller-এ কোনো build চলে না — শুধু পরিচালনা

সম্পূর্ণ কনফিগারেশন

```
# পদ্ধতি A — Docker agent (ল্যাবের জন্য সুপারিশকৃত, Jenkinsfile-এ লেখা হয় — ল্যাব ১৯ দেখুন):
#   agent { docker { image 'node:20-alpine' } }
# পদ্ধতি B — স্থায়ী agent (Manage Jenkins → Nodes → New Node):
#   Name: node-agent | Type: Permanent Agent
#   Remote root directory: /home/jenkins/agent
#   Labels: node20
#   Launch method: Launch agent by connecting it to the controller (inbound/JNLP)
#   → Secret পাওয়া যায় — agent মেশিনে agent.jar চালিয়ে সংযোগ করুন:
#   java -jar agent.jar -jnlpUrl http://203.0.113.10:8080/computer/node-agent/jenkins-
agent.jnlp \
#     -secret CHANGE-ME-SECRET -workDir "/home/jenkins/agent"
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: Jenkins-এ Manage Jenkins → Nodes-এ গিয়ে built-in node-এ “0 executors” আছে কি না দেখুন (controller-এ build বন্ধ রাখাই ভালো অভ্যাস — ল্যাব ১৭-এর job-টি এখানে আটকে থাকলে বুঝবেন কেন)। **ধাপ ২:** পদ্ধতি B-তে `node-agent` তৈরি করুন ও secret নিন। **ধাপ ৩:** Agent মেশিনে (দ্বিতীয় VM বা একই VM-এর আলাদা ফোল্ডার) `agent.jar` ডাউনলোড করে উপরের কমান্ডে সংযোগ করুন — Nodes পাতায় agent “online” হওয়া দেখুন। **ধাপ ৪:** একটি Freestyle job তৈরি করে “Restrict where this project can be run” → Label: `node20` দিন, build চালান — Console Output-এ “Running on node-agent” দেখুন। **ধাপ ৫:** Agent সংযোগ বিচ্ছিন্ন করে আবার build চালান — job অপেক্ষায় (pending) থাকা দেখুন।

যাচাই ও প্রত্যাশিত ফলাফল

Nodes পাতায় agent online; লেবেল করা job agent-এ চলে ও log-এ agent-এর নাম দেখা যায়; agent offline থাকলে job pending থাকে — controller নিজে build নেয় না।

সাধারণ ব্যর্থতা ও সমাধান

Agent সংযোগ হচ্ছে না: Controller-এর URL agent মেশিন থেকে খোলা যায় কি না, secret ঠিক কি না, পোর্ট 50000 খোলা কি না দেখুন। **Job ভুল node-এ চলছে:** Label বানান মিলছে কি না দেখুন — label case-sensitive। **Agent-এ Node নেই:** Agent মেশিনে Node.js 20 ইনস্টল আছে কি না দেখুন, অথবা Docker agent ব্যবহার করুন।

Cleanup

Agent সংযোগ বিচ্ছিন্ন করুন ও Nodes পাতা থেকে মুছে ফেলুন। Controller-এর executor সংখ্যা 0-তে রেখে দিন — প্রোডাকশনে এটাই নিরাপদ ডিফল্ট।

Reflection প্রশ্ন

১. Controller-এ build চালালে কী কী ঝুঁকি (নিরাপত্তা, স্থিতি)? ২. Label দিয়ে job রাউট করা কেন দরকার? ৩. Agent offline হলে pipeline কীভাবে আচরণ করা উচিত — অপেক্ষা, নাকি ব্যর্থ?

ল্যাব 19

একটি Declarative Jenkins Pipeline তৈরি করা

উদ্দেশ্য (Objective): Jenkinsfile লিখে একটি সম্পূর্ণ CI pipeline বানানো — checkout → install → lint → test → build — এবং প্রতিটি stage-এর ফল Stage View-এ দেখা। Jenkinsfile কোড হিসেবে থাকে বলে pipeline-এর ইতিহাস Git-এ থাকে — এটাই Pipeline-as-Code।

পূর্বশর্ত (Prerequisites): ল্যাব ১৭–১৮ সম্পূর্ণ; GitHub-এ Node অ্যাপ আছে। **প্রয়োজনীয় টুল:** Jenkins, Git, Node.js 20 (Docker agent-এ)।

আর্কিটেকচার:
Git push → Jenkins Pipeline job (Jenkinsfile from SCM) → Agent: docker node:20 → Stages: Checkout → Install → Lint → Test → Build → Post: সবসময় workspace পরিষ্কার

সম্পূর্ণ কনফিগারেশন

```
// Jenkinsfile — রিপোজিটরির root-এ রাখুন
pipeline {
  agent { docker { image 'node:20-alpine' } }
  options { timestamps() }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Install') {
      steps { sh 'npm ci' }
    }
    stage('Lint') {
      steps { sh 'npm run lint' }
    }
    stage('Test') {
      steps { sh 'npm test' }
    }
    stage('Build') {
      steps { sh 'npm run build' }
    }
  }
  post {
    always { echo 'Pipeline finished — cleaning workspace' }
    failure { echo 'Pipeline FAILED — check the failing stage' }
    success { echo 'Pipeline SUCCESS' }
  }
}
// Jenkins-এ job তৈরি: New Item → Pipeline → নাম technova-pipeline
//   Definition: Pipeline script from SCM → SCM: Git → Repository URL: https://github.com/
<owner>/technova.git
//   Script Path: Jenkinsfile → Save → Build Now
```

Field ব্যাখ্যা: `agent { docker {...} }` — প্রতিটি build নতুন container-এ চলে; `stages` — ধাপের ক্রম; `post` — ফল অনুযায়ী সবসময়/ব্যর্থতায়/সাফল্যে চলা ধাপ; `checkout scm` — job-এর Git থেকে কোড নেয়।

ধাপে ধাপে নির্দেশনা

ধাপ ১: রিপোজিটরির root-এ `Jenkinsfile` তৈরি করে push করুন। **ধাপ ২:** Jenkins-এ Pipeline job তৈরি করুন (উপরের কনফিগারেশন দেখুন) — প্রথম build-এ SCM credential লাগলে public টেস্ট রিপোজিটরি ব্যবহার করুন অথবা credential যোগ করুন (ল্যাব ১৭-এর Jenkins-এ Manage Jenkins → Credentials)। **ধাপ ৩:** Build Now চাপুন — Stage View-এ প্রতিটি stage সবুজ হওয়া দেখুন। **ধাপ ৪:** Test ভেঙে push করুন — Pipeline লাল হয়, পরের stage চলে না, `post.failure` বার্তা দেখা যায়। **ধাপ ৫:** ঠিক করে আবার build করুন।

যাচাই ও প্রত্যাশিত ফলাফল

Stage View-এ সব stage সবুজ; Console Output-এ প্রতিটি stage-এর log আলাদা; ভাঙা test-এ pipeline সেই stage-এ থামে; build history-তে প্রতিটি build-এর নম্বর ও সময় থাকে।

সাধারণ ব্যর্থতা ও সমাধান

Jenkinsfile পাওয়া যাচ্ছে না: Script Path `Jenkinsfile` ও branch ঠিক কি না দেখুন। **Docker agent ব্যর্থ:** Jenkins-কে Docker socket-এ অধিকার আছে কি না দেখুন — ল্যাব Docker-এ চালালে `-v /var/run/docker.sock:/var/run/docker.sock` লাগতে পারে (এটি ল্যাবের জন্য, প্রোডাকশনে ঝুঁকিপূর্ণ — অধ্যায় ১৭ দেখুন)। **SCM checkout ব্যর্থ:** Repository URL ও credential মিলিয়ে দেখুন।

Cleanup

টেস্ট build-গুলোর workspace Jenkins নিজেই পরিষ্কার করে — তবুও পুরনো build মুছতে job পাতা থেকে “Delete” ব্যবহার করুন। Jenkinsfile রিপোজিটরিতে রেখে দিন।

Reflection প্রশ্ন

১. Jenkinsfile Git-এ রাখার সুবিধা কী (ইতিহাস, review, rollback)? ২. Declarative আর Scripted pipeline — কখন কোনটি? ৩. `post` block-এ notification পাঠানো কেন গুরুত্বপূর্ণ?

ল্যাব 20

Automated Testing ও Linting একীভূত করা

উদ্দেশ্য (Objective): CI pipeline-এ lint, type check, unit test ও coverage — চারটি quality gate একসাথে বসানো। কোনো gate ব্যর্থ হলে merge বন্ধ থাকবে। প্রতিটি gate-এর সীমাও বুঝুন — সবুজ test মানেই নির্ভুল কোড নয়।

পূর্বশর্ত (Prerequisites): ল্যাব ২ ও ৫ সম্পূর্ণ; Node অ্যাপে Jest/Vitest অথবা Node-এর বিল্ট-ইন test runner আছে। **প্রয়োজনীয় টুল:** GitHub Actions, ESLint, Node.js 20।

আর্কিটেকচার:
PR → CI: Job test → Steps: ESLint (শৈলী ও ভুল) → Type check (TypeScript থাকলে) → Unit test (Jest/Vitest) → Coverage report (artifact হিসেবে) → কোনো step লাল হলে PR merge বন্ধ

সম্পূর্ণ কনফিগারেশন

```
# package.json-এ script (উদাহরণ):
#   "lint": "eslint .",
#   "test": "node --test",
#   "test:coverage": "node --test --experimental-test-coverage"
# .github/workflows/ci.yml — test job
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run test:coverage
        continue-on-error: true
      - uses: actions/upload-artifact@v4
        with: { name: coverage-report, path: coverage/, retention-days: 7 }
```

Field ব্যাখ্যা: `continue-on-error: true` — coverage step ব্যর্থ হলেও pipeline থামবে না (coverage তথ্য হিসেবে, gate হিসেবে নয় — যতক্ষণ না আপনি সীমা নির্ধারণ করেন); coverage artifact — রিপোর্ট পরে পড়া যায়।

ধাপে ধাপে নির্দেশনা

ধাপ ১: ESLint ইনস্টল করুন (`npm install --save-dev eslint` ও `npx eslint --init` — ল্যাবের জন্য ডিফল্ট নিয়মই যথেষ্ট)। **ধাপ ২:** উপরের workflow লিখে push করুন। **ধাপ ৩:** একটি lint ভুল (যেমন অব্যবহৃত ভ্যারিয়েবল) দিয়ে PR খুলুন — CI লাল হওয়া দেখুন। **ধাপ ৪:** ভুল ঠিক করে আবার push করুন — সবুজ হওয়া দেখুন। **ধাপ ৫:** Coverage artifact ডাউনলোড করে দেখুন কোন ফাইলের coverage কম। **ধাপ ৬:** একটি test লিখুন যা পাস করে কিন্তু ভুল আচরণ ধরে না (যেমন শুধু `true` assert করে) — লিখুন কেন coverage বেশি হলেও test দুর্বল হতে পারে।

যাচাই ও প্রত্যাশিত ফলাফল

Lint ভুলে CI লাল; ঠিক করলে সবুজ; coverage artifact-এ প্রতিটি ফাইলের শতাংশ দেখা যায়; দুর্বল test-এর উদাহরণ নোটে লেখা থাকে।

সাধারণ ব্যর্থতা ও সমাধান

ESLint config পাওয়া যাচ্ছে না: `eslint.config.js` (flat config) আছে কি না দেখুন — ESLint 9-এ পুরনো `.eslintrc` চলে না। **Coverage step ব্যর্থ:** Node সংস্করণে `--experimental-test-coverage` আছে কি না দেখুন — না থাকলে step-টি `continue-on-error`-এ রাখুন। **সব test পাস কিন্তু bug আছে:** এটাই test-এর সীমা — test শুধু লেখা assertion যাচাই করে; review ও manual যাচাই এখনো দরকার।

Cleanup

Coverage ফোল্ডার `.gitignore`-এ রাখুন। ESLint config রিপোজিটরিতে রেখে দিন।

Reflection প্রশ্ন

১. Lint, type check, unit test — প্রতিটি কী ধরে, কী ধরে না? ২. Coverage 100% হলেই কি কোড নির্ভুল? কেন নয়? ৩. কোন gate-টি `continue-on-error`-এ রাখা উচিত, কোনটি কখনো নয়?

ল্যাব 21

Docker Image Build ও Publish করা

উদ্দেশ্য (Objective): MERN অ্যাপের জন্য multi-stage Dockerfile লিখে image build করা, সংস্করণ tag দেওয়া ও GitHub Container Registry (GHCR)-এ publish করা। Image-ই deploy-যোগ্য artifact — `latest` tag-এর ঝুঁকি ও digest-এর স্থিরতা নিজে দেখুন।

পূর্বশর্ত (Prerequisites): Docker Engine চালু; Node অ্যাপ; GitHub অ্যাকাউন্ট। **প্রয়োজনীয় টুল:** Docker, GitHub Actions, GHCR।

আর্কিটেকচার:

Dockerfile (multi-stage: build → runtime) → docker build → Image: ghcr.io/<owner>/technova:v1.2.3 (+ digest sha256:...) → Push to GHCR → Deploy job image digest দিয়ে টানে (tag নয়)

সম্পূর্ণ কনফিগারেশন

```
# Dockerfile — multi-stage (server-এর জন্য)
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build
FROM node:20-alpine AS runtime
WORKDIR /app
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package*.json ./
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]

# .github/workflows/docker.yml
name: Docker
on: { push: { tags: ['v*'] } }
permissions: { contents: read, packages: write }
jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
```

```
          push: true
          tags: ghcr.io/${{ github.repository }}/technova:${{ github.ref_name }}
```

Field ব্যাখ্যা: Multi-stage — build stage-এর dev dependency runtime image-এ যায় না, image ছোট ও নিরাপদ হয়; `USER node` — root হিসেবে চালানো হয় না; `packages: write` — GHCR-এ push-এর জন্য সর্বনিম্ন অনুমতি; `push: tags v*` — শুধু version tag-এ image publish হয়।

ধাপে ধাপে নির্দেশনা

ধাপ ১: Dockerfile লিখে স্থানীয়ভাবে build করুন: `docker build -t technova:lab .` ও `docker images technova` দিয়ে সাইজ দেখুন। **ধাপ ২:** Container চালান: `docker run -p 3000:3000 technova:lab` ও `curl http://localhost:3000` দিয়ে যাচাই করুন। **ধাপ ৩:** Workflow ফাইল লিখে `git tag v0.1.0-lab` ও push করুন — GHCR-এ image আসা দেখুন। **ধাপ ৪:** `docker pull ghcr.io/<owner>/technova:v0.1.0-lab` দিয়ে টেনে আবার চালান। **ধাপ ৫:** Image-এর digest লিখে নিন (GitHub Packages পাতায় দেখা যায়) — tag বদলালেও digest একই থাকে কি না, ভাবুন।

যাচাই ও প্রত্যাশিত ফলাফল

স্থানীয় container `curl`-এ সাড়া দেয়; GHCR-এ image ও tag দেখা যায়; digest লেখা থাকে; pull করে আবার চালানো যায়।

সাধারণ ব্যর্থতা ও সমাধান

Build ব্যর্থ (COPY পাওয়া যাচ্ছে না): `.dockerignore`-এ `node_modules` ও `.git` আছে কি না, Dockerfile-এর পথ ঠিক কি না দেখুন। **Push-এ 403:** `permissions.packages: write` আছে কি না, রিপোজিটরির Packages সেটিং দেখুন। **Container সঙ্গে সঙ্গে বন্ধ:** `docker logs <id>` দেখুন — `dist/server.js` পথ ঠিক কি না, পোর্ট ঠিক কি না দেখুন।

Cleanup

স্থানীয় image মুছুন: `docker rmi technova:lab`; চলমান container থামান: `docker stop`। GHCR-এর ল্যাব image রেখে দিতে পারেন অথবা মুছে ফেলুন।

Reflection প্রশ্ন

১. `latest` tag দিয়ে deploy করলে rollback-এ কী সমস্যা হয়? ২. Multi-stage build image-কে কীভাবে ছোট ও নিরাপদ করে? ৩. `USER node` না দিলে কী ঝুঁকি থাকে?

ল্যাব 22

Dependency ও Image নিরাপত্তা যাচাই যোগ করা

উদ্দেশ্য (Objective): Pipeline-এ dependency vulnerability scan ও container image scan বসানো — যাতে পরিচিত দুর্বলতা (vulnerability) থাকা dependency বা image deploy-এর আগেই ধরা পড়ে। Scan সবুজ মানেই নিরাপদ নয় — সীমাটাও বুঝুন।

পূর্বশর্ত (Prerequisites): ল্যাব ২০–২১ সম্পূর্ণ; Docker image আছে। **প্রয়োজনীয় টুল:** GitHub Actions, npm audit, Trivy (অথবা Docker Scout)।

আর্কিটেকচার:
PR → CI: npm audit (dependency) → Trivy scan (image) → ফল: SARIF রিপোর্ট GitHub Security ট্যাবে → Critical দুর্বলতা থাকলে pipeline লাল → ঠিক করে আবার scan

সতর্কতা: Scan-এর ফলাফলে দুর্বলতার নাম ও সংস্করণ থাকতে পারে — সেটি টেস্ট রিপোজিটরির জন্য ঠিক আছে, কিন্তু প্রোডাকশন দুর্বলতার বিবরণ কখনো প্রকাশ্যে শেয়ার করবেন না। Scan রিপোর্টে secret থাকলে সেটি revoke করুন।

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/security.yml
name: Security Scan
on: { push: { branches: [main] }, pull_request: { branches: [main] } }
permissions: { contents: read, security-events: write }
jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm audit --audit-level=high
        continue-on-error: true
      - name: Build image for scan
        run: docker build -t technova:scan .
      - uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: technova:scan
          format: sarif
          output: trivy-results.sarif
          severity: HIGH,CRITICAL
      - uses: github/codeql-action/upload-sarif@v3
        with: { sarif_file: trivy-results.sarif }
```

Field ব্যাখ্যা: `npm audit --audit-level=high` — high ও critical দুর্বলতা দেখায়; `trivy-action` — image-এর OS প্যাকেজ ও dependency স্ক্যান করে; `severity: HIGH,CRITICAL` — শুধু গুরুতর দুর্বলতায় ব্যর্থ হয়; SARIF upload — ফল GitHub Security → Code scanning-এ দেখা যায়।

ধাপে ধাপে নির্দেশনা

ধাপ ১: স্থানীয়ভাবে `npm audit` চালিয়ে দেখুন কী কী দুর্বলতা আছে (ল্যাব অ্যাপে থাকতে পারে — সেটাই স্বাভাবিক)। **ধাপ ২:** উপরের workflow লিখে push করুন। **ধাপ ৩:** Security ট্যাবে Trivy-র ফল দেখুন — কোন প্যাকেজ, কোন সংস্করণ, কতটা গুরুতর। **ধাপ ৪:** একটি দুর্বল dependency আপডেট করুন (`npm audit fix` — ল্যাবের জন্য), আবার scan চালান — ফল কমা দেখুন। **ধাপ ৫:** ইচ্ছা করে একটি পুরনো দুর্বল সংস্করণ বসিয়ে দেখুন pipeline লাল হয় কি না (তারপর সরিয়ে ফেলুন)।

যাচাই ও প্রত্যাশিত ফলাফল

Security ট্যাবে scan ফল দেখা যায়; দুর্বলতা ঠিক করলে ফল কমে; গুরুতর দুর্বলতায় pipeline লাল হয়।

সাধারণ ব্যর্থতা ও সমাধান

`npm audit` ব্যর্থ হলেও pipeline সবুজ: `continue-on-error: true` থাকলে সেটাই হবে — gate হিসেবে চাইলে সেটি সরিয়ে `--audit-level=high`-কে ব্যর্থ হতে দিন। **Trivy action সংস্করণ:** Action-টি পিন করা সংস্করণে (`@0.28.0`) ব্যবহার করুন — ভাসমান সংস্করণ নিরাপত্তা ঝুঁকি। **False positive:** Scan-এর প্রতিটি সতর্কতা সত্যি নয় — CVE বিবরণ পড়ে সিদ্ধান্ত নিন, অন্ধভাবে নয়।

Cleanup

পরীক্ষার জন্য বসানো দুর্বল সংস্করণটি সরিয়ে ফেলুন। Scan করা image মুছুন: `docker rmi technova:scan`।

Reflection প্রশ্ন

১. Scan সবুজ হলেই কি অ্যাপ নিরাপদ? কী কী scan ধরে না (যেমন ব্যবসায়িক যুক্তির ভুল)? ২. `npm audit fix` কখন ভাঙা পরিবর্তন আনতে পারে? ৩. Third-party action পিন করা কেন নিরাপত্তার অংশ?

ল্যাব 23

অনুমোদনসহ Staging-এ Deploy করা

উদ্দেশ্য (Objective): Staging environment-এ deploy-এর আগে অনুমোদন (approval) বসানো — যাতে main-এ merge হলেই স্বয়ংক্রিয়ভাবে staging বদলে না যায়, একজন মানুষ দেখে অনুমোদন দেয়। এটাই deployment gate-এর মূল।

পূর্বশর্ত (Prerequisites): ল্যাব ১২ ও ২১ সম্পূর্ণ; Docker image GHCR-এ আছে। **প্রয়োজনীয় টুল:** GitHub Actions, Docker, একটি staging সার্ভার বা ল্যাব VM।

আর্কিটেকচার:

Tag v* push → CI (test + build + scan) → Job: deploy-staging (environment: staging, required reviewers: 1) → ⏸️ Waiting for approval → অনুমোদন → docker pull + docker run (staging.technova.example) → Health check

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/deploy-staging.yml
name: Deploy to Staging
on: { push: { tags: ['v*'] } }
permissions: { contents: read, packages: read }
jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.technova.example
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to staging (lab — simulated on runner)
        run: |
          echo "Pulling ghcr.io/${{ github.repository }}/technova:${{ github.ref_name }}"
          echo "Deploying to staging.technova.example (lab simulation)"
          echo "Health check: curl -f https://staging.technova.example/health || echo 'staging
host not present in lab — check skipped'"
# GitHub UI: Settings → Environments → staging → Required reviewers: 1 জন যোগ করুন
# আসল সার্ভারে deploy করতে SSH ধাপ লাগে — ল্যাবে simulation-ই যথেষ্ট; প্রোডাকশনে
# SSH key GitHub Secret-এ রাখুন ও appleboy/ssh-action-এর মতো পিন করা action ব্যবহার করুন।
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: Staging environment-এ Required reviewers যোগ করুন (ল্যাব ১২-এর মতো)। **ধাপ ২:** Workflow লিখে `v0.2.0-lab` tag push করুন। **ধাপ ৩:** Actions ট্যাবে deploy job “Waiting for approval” অবস্থায় থাকা দেখুন — অনুমোদন না দিলে job চলবে না, সময় নিয়ে দেখুন। **ধাপ ৪:** অনুমোদন দিন — job চলা ও সবুজ হওয়া দেখুন। **ধাপ ৫:** আবার একটি tag push করে এবার “Reject” চাপুন — deploy না চলা দেখুন।

যাচাই ও প্রত্যাশিত ফলাফল

অনুমোদন ছাড়া deploy job শুরু হয় না; অনুমোদনে চলে; প্রত্যাখ্যানে চলে না; প্রতিটি deploy-এর ইতিহাস Actions ট্যাবে থাকে (কে অনুমোদন দিল, কখন)।

সাধারণ ব্যর্থতা ও সমাধান

Approval বোতাম আসছে না: নিজের run নিজে অনুমোদন করা যায় না — ল্যাবে reviewer হিসেবে অন্য অ্যাকাউন্ট ব্যবহার করুন অথবা reviewer নিয়ম সাময়িক শিথিল করুন। **Job সঙ্গে সঙ্গে চলছে:** Environment-এর নাম YAML-এর সঙ্গে হুবহু মিলছে কি না দেখুন। **Staging URL খুলছে না:** ল্যাবে simulation-এ URL আসল হোস্ট নয় — health check-এর ফল “skipped” হওয়াই প্রত্যাশিত।

Cleanup

ল্যাব tag-গুলো মুছে ফেলুন (`git tag -d v0.2.0-lab` ও remote থেকে মুছুন)। Staging environment-এর reviewer নিয়ম রেখে দিন।

Reflection প্রশ্ন

১. Staging-এ অনুমোদন কেন দরকার, যদি CI ইতিমধ্যে সবুজ হয়? ২. অনুমোদনকারী কী কী দেখে সিদ্ধান্ত নেবেন (test ফল, scan, পরিবর্তনের তালিকা)? ৩. অনুমোদন প্রক্রিয়া কখন বোতল-গলা (bottleneck) হয়ে যায়, এবং কীভাবে সামলাবেন?

ল্যাব 24

Deployment যাচাই ও Rollback করা

উদ্দেশ্য (Objective): Deploy-এর পর health check দিয়ে যাচাই করা, ভাঙা deploy শনাক্ত করা ও আগের সংস্করণে rollback করা। Deploy শেষ হওয়া মানেই সফল deploy নয় — যাচাই (verification) ছাড়া delivery অসম্পূর্ণ।

পূর্বশর্ত (Prerequisites): ল্যাব ২১ ও ২৩ সম্পূর্ণ; দুটি সংস্করণের image আছে (v0.1.0-lab ও v0.2.0-lab)। **প্রয়োজনীয় টুল:** Docker, GitHub Actions, curl।

আর্কিটেকচার:
Deploy v0.2.0 → Health check (/health: 200 OK?) → ✅ সফল: deploy সম্পূর্ণ → ❌ ব্যর্থ: Rollback → আগের image v0.1.0 আবার চালু → আবার Health check → ✅ সেবা ফিরে আসে
Rollback = আগের পরিচিত-ভালো সংস্করণে ফেরা, নতুন করে ঠিক করা নয়

সম্পূর্ণ কনফিগারেশন

```
# scripts/verify.sh — deploy যাচাই (স্থানীয়ভাবে ও pipeline-এ একই script)
#!/usr/bin/env bash
set -euo pipefail
URL="${1:-http://localhost:3000/health}"
echo "Checking $URL ..."
for i in 1 2 3 4 5; do
  if curl -fsS "$URL" >/dev/null 2>&1; then echo "HEALTH OK"; exit 0; fi
  echo "Attempt $i failed — retrying in 3s..."; sleep 3
done
echo "HEALTH CHECK FAILED"; exit 1

# scripts/rollback.sh — আগের সংস্করণে ফেরা (ল্যাব simulation)
#!/usr/bin/env bash
set -euo pipefail
PREVIOUS="${1:-v0.1.0-lab}"
echo "Rolling back to $PREVIOUS ..."
docker stop technova-app || true
docker rm technova-app || true
docker run -d --name technova-app -p 3000:3000 "ghcr.io/OWNER/technova:$PREVIOUS"
./scripts/verify.sh
echo "Rollback complete — $PREVIOUS is live"
```

Field ব্যাখ্যা: Health check ৫ বার চেষ্টা করে — অ্যাপ চালু হতে সময় লাগে বলে একবারের ব্যর্থতায় rollback নয়; `curl -f` — HTTP error-এ ব্যর্থ হয়; rollback script আগের tag-টি parameter হিসেবে নেয়, হার্ডকোড করে না।

ধাপে ধাপে নির্দেশনা

ধাপ ১: `scripts/verify.sh` ও `scripts/rollback.sh` তৈরি করুন, `chmod +x` দিন। **ধাপ ২:** অ্যাপে `/health` endpoint যোগ করুন (Express-এ `app.get('/health', (req,res) => res.json({status:'ok'}))`)। **ধাপ ৩:** v0.2.0-lab image চালিয়ে `./scripts/verify.sh` দিয়ে যাচাই করুন — “HEALTH OK” দেখুন। **ধাপ ৪:** ইচ্ছা করে ভাঙা সংস্করণ চালান (যেমন ভুল পোর্টে) — health check ব্যর্থ হওয়া দেখুন। **ধাপ ৫:** `./scripts/rollback.sh`

`v0.1.0-lab` চালান — আগের সংস্করণ চালু ও health check সবুজ হওয়া দেখুন। **ধাপ ৬:** পুরো ঘটনা একটি incident নোটে লিখুন: কী ভেঙেছিল, কীভাবে ধরা পড়ল, rollback কত সময় নিল।

যাচাই ও প্রত্যাশিত ফলাফল

ভালো deploy-এ “HEALTH OK”; ভাঙা deploy-এ ৫ চেষ্টার পর “HEALTH CHECK FAILED”; rollback-এর পর আগের সংস্করণ চালু ও health check সবুজ; incident নোটে সময়সহ ঘটনা লেখা থাকে।

সাধারণ ব্যর্থতা ও সমাধান

Health check সবসময় ব্যর্থ: `/health` endpoint আছে কি না, পোর্ট মিলছে কি না, container log (`docker logs technova-app`) দেখুন। **Rollback-এ পুরনো image নেই:** Image ট্যাগ মুছে ফেললে rollback অসম্ভব — তাই পুরনো image সংরক্ষণ করুন (retention নীতি দেখুন)। **Rollback করেও সেবা ফিরছে না:** সমস্যা ডেটাবেস migration-এ হতে পারে — code rollback ডেটা rollback নয় (অধ্যায় ২২ দেখুন)।

Cleanup

চলমান container থামান ও মুছুন (`docker stop technova-app && docker rm technova-app`)। ল্যাব image-গুলো রেখে দিতে পারেন অথবা মুছে ফেলুন।

Reflection প্রশ্ন

১. Health check কী কী পরীক্ষা করা উচিত (শুধু “সার্ভার চালু” যথেষ্ট কেন নয়)? ২. Rollback কখন যথেষ্ট, আর কখন forward fix দরকার? ৩. Rollback-এর সময় ডেটাবেস migration থাকলে কী করবেন?

অ্যাসাইনমেন্ট

ব্যবহারিক অ্যাসাইনমেন্ট — ১৫টি

প্রতিটি অ্যাসাইনমেন্ট একা করুন, নিজের টেস্ট রিপোজিটরিতে। জমা দেওয়ার জিনিস (deliverable) একটি লিখিত নোট + রিপোজিটরির লিংক বা স্ক্রিনশট নয় — কাজ করা pipeline, তার log ও আপনার সিদ্ধান্তের ব্যাখ্যা। প্রতিটি অ্যাসাইনমেন্টের মূল্যায়ন মানদণ্ড (evaluation criteria) দেওয়া আছে; নিজে নিজে নম্বর দিন, তারপর একজন সহপাঠীকে দিয়ে যাচাই করান।

অ্যাসাইনমেন্ট ১ — Manual Delivery-র খরচ মাপা

কাজ (Task): আপনার MERN অ্যাপের একটি পরিবর্তন manual deploy (SSH + git pull + install + restart) দিয়ে সার্ভারে তুলুন, প্রতিটি ধাপের সময় ও ভুলের ঝুঁকি লিখুন। তারপর একই পরিবর্তন একটি script দিয়ে deploy করুন ও তুলনা করুন।

জমা (Deliverable): দুই পদ্ধতির সময়-তালিকা, ঝুঁকি-তালিকা ও এক পৃষ্ঠার সুপারিশ — কোন ধাপটি প্রথমে automate করা উচিত, এবং কেন।

মূল্যায়ন: সময় মাপা হয়েছে কি না (৩০%), প্রতিটি ধাপের ঝুঁকি চিহ্নিত কি না (৪০%), সুপারিশ যুক্তিসংগত কি না (৩০%)।

অ্যাসাইনমেন্ট ২ — PR-ভিত্তিক CI ডিজাইন

কাজ: একটি টিমের জন্য PR workflow ডিজাইন করুন — কোন check বাধ্যতামূলক, কোনটি ঐচ্ছিক, কে review করবে, কখন merge করা যাবে। GitHub-এ branch protection দিয়ে বাস্তবায়ন করুন।

জমা: Workflow ডায়াগ্রাম, branch protection-এর স্ক্রিনশট-বিবরণ ও নিয়মের যুক্তি (প্রতিটি নিয়ম কেন)।

মূল্যায়ন: বাধ্যতামূলক check সম্পূর্ণ কি না (৪০%), নিয়মের যুক্তি স্পষ্ট কি না (৩০%), ভাঙা PR-এ merge বন্ধ হওয়ার প্রমাণ আছে কি না (৩০%)।

অ্যাসাইনমেন্ট ৩ — Trigger নীতি লেখা

কাজ: একটি মাঝারি রিপোজিটরির জন্য trigger নীতি লিখুন — main, feature, release branch ও docs পরিবর্তনের জন্য আলাদা নিয়ম, paths ফিল্টারসহ। বাস্তবায়ন করে প্রতিটি নিয়মের পরীক্ষা দেখান।

জমা: নীতি-নথি (এক পৃষ্ঠা) + `ci.yml` + প্রতিটি trigger-এর run-এর ফল।

মূল্যায়ন: অপ্রয়োজনীয় run কমেছে কি না (৪০%), প্রতিটি নিয়ম পরীক্ষিত কি না (৪০%), paths ফিল্টারের ঝুঁকি উল্লেখ আছে কি না (২০%)।

অ্যাসাইনমেন্ট ৪ — Job বিভাজন ও নির্ভরতা

কাজ: একটি একক-job workflow-কে test, build ও package — তিনটি job-এ ভাগ করুন, `needs` দিয়ে নির্ভরতা বসান, এবং ব্যর্থ test-এ পরের job না চলার প্রমাণ দেখান।

জমা: নতুন workflow, job-চিত্র ও ব্যর্থতার পরীক্ষার log-বিবরণ।

মূল্যায়ন: নির্ভরতা সঠিক কি না (৪০%), ব্যর্থতায় আচরণ সঠিক কি না (৩০%), job বিভাজনের যুক্তি লেখা আছে কি না (৩০%)।

অ্যাসাইনমেন্ট ৫ — Runner নির্বাচন প্রতিবেদন

কাজ: GitHub-hosted ও self-hosted runner-এর তুলনা লিখুন — খরচ, গতি, নিরাপত্তা, রক্ষণাবেক্ষণ — এবং আপনার অ্যাপের জন্য কোনটি, কখন, সেই সুপারিশ দিন। Self-hosted runner একটি ল্যাব VM-এ চালিয়ে পরীক্ষা করুন।

জমা: তুলনা-টেবিল (অন্তত ৬টি মাপকাঠি) + সুপারিশ + self-hosted পরীক্ষার ফল।

মূল্যায়ন: তুলনা সম্পূর্ণ কি না (৪০%), সুপারিশ প্রমাণ-ভিত্তিক কি না (৩০%), নিরাপত্তা ঝুঁকি উল্লেখ আছে কি না (৩০%)।

অ্যাসাইনমেন্ট ৬ — Reusable Workflow লাইব্রেরি

কাজ: তিনটি workflow-এর পুনরাবৃত্ত ধাপ থেকে একটি reusable workflow বানান (test + lint), দুটি caller workflow থেকে ডাকুন, এবং input দিয়ে Node সংস্করণ বদলে পরীক্ষা করুন।

জমা: Reusable workflow ফাইল, দুটি caller ফাইল ও পরীক্ষার ফল।

মূল্যায়ন: পুনরাবৃত্তি দূর হয়েছে কি না (৪০%), input কাজ করে কি না (৩০%), সংস্করণ-পরিবর্তনের ঝুঁকি নিয়ে নোট আছে কি না (৩০%)।

অ্যাসাইনমেন্ট ৭ — Composite Action বনাম Reusable Workflow

কাজ: একই কাজ (Node setup + install + lint) composite action ও reusable workflow — দুভাবেই বাস্তবায়ন করুন, এবং একটি সিদ্ধান্ত-নিয়ম লিখুন: কখন কোনটি ব্যবহার করবেন।

জমা: দুটি বাস্তবায়ন + এক পৃষ্ঠার সিদ্ধান্ত-নিয়ম ও উদাহরণ।

মূল্যায়ন: দুটি বাস্তবায়নই চলে কি না (৪০%), পার্থক্য সঠিকভাবে ব্যাখ্যা করা হয়েছে কি না (৪০%), সিদ্ধান্ত-নিয়ম ব্যবহারযোগ্য কি না (২০%)।

অ্যাসাইনমেন্ট ৮ — Matrix কৌশল ও খরচ

কাজ: Node 18/20/22 ও Ubuntu/Windows — এই matrix-এর মোট job সংখ্যা ও আনুমানিক সময় হিসাব করুন, তারপর বাস্তবে একটি ছোট matrix চালিয়ে সময় মাপুন ও খরচ কমানোর পরিকল্পনা লিখুন (যেমন PR-এ ছোট matrix, main-এ বড়)।

জমা: হিসাব-টেবিল, বাস্তব run-এর সময় ও খরচ-কমানোর পরিকল্পনা।

মূল্যায়ন: হিসাব সঠিক কি না (৩০%), বাস্তব মাপ আছে কি না (৪০%), পরিকল্পনা বাস্তবসম্মত কি না (৩০%)।

অ্যাসাইনমেন্ট ৯ — Cache নীতি ও পরিমাপ

কাজ: Cache ছাড়া ও cache-সহ workflow-এর সময় তিনবার করে মাপুন, গড় বের করুন, এবং cache key নীতি লিখুন (key-তে কী থাকবে, কখন invalidate হবে)।

জমা: পরিমাপ-টেবিল, গড় সময় ও cache নীতি-নথি।

মূল্যায়ন: পরিমাপ পদ্ধতি সঠিক কি না (৪০%), key নীতি নিরাপদ কি না (৪০%), সিদ্ধান্ত প্রমাণ-ভিত্তিক কি না (২০%)।

অ্যাসাইনমেন্ট ১০ — Artifact কৌশল

কাজ: Build artifact, test রিপোর্ট ও coverage রিপোর্ট — তিন ধরনের artifact-এর জন্য নাম, পথ ও retention period নির্ধারণ করুন, বাস্তবায়ন করুন, এবং retention-এর খরচ-যুক্তি লিখুন।

জমা: Artifact-নীতি টেবিল + workflow + ডাউনলোড করা artifact-এর যাচাই।

মূল্যায়ন: তিন ধরনের artifact-ই আছে কি না (৩০%), retention যুক্তিসংগত কি না (৪০%), secret artifact-এ যাচ্ছে না — সেই যাচাই আছে কি না (৩০%)।

অ্যাসাইনমেন্ট ১১ — Secret ও Environment নকশা

কাজ: Staging ও production-এর জন্য আলাদা secret ও environment ডিজাইন করুন — কোন secret কোথায়, কার অনুমোদন লাগবে, rotation কতদিন পর। কাল্পনিক মান দিয়ে বাস্তবায়ন করুন।

জমা: Secret-নকশা টেবিল, environment সেটিং-এর বিবরণ ও rotation পরিকল্পনা।

মূল্যায়ন: পরিবেশ আলাদা কি না (৩০%), least privilege মানা হয়েছে কি না (৪০%), rotation পরিকল্পনা আছে কি না (৩০%)।

অ্যাসাইনমেন্ট ১২ — GitLab বনাম GitHub তুলনা

কাজ: একই অ্যাপের pipeline GitHub Actions ও GitLab CI — দুটোতেই বাস্তবায়ন করুন, এবং স্থাপত্য, YAML, runner, artifact, environment ও ডিবাগ-অভিজ্ঞতা — এই ছয়টি দিকে তুলনা লিখুন।

জমা: দুটি pipeline ফাইল + তুলনা-টেবিল + আপনার পছন্দ ও তার কারণ।

মূল্যায়ন: দুটি pipeline-ই চলে কি না (৪০%), তুলনা ছয়টি দিকেই আছে কি না (৪০%), পছন্দের কারণ প্রমাণ-ভিত্তিক কি না (২০%)।

অ্যাসাইনমেন্ট ১৩ — Jenkins Pipeline-as-Code

কাজ: একই অ্যাপের জন্য Jenkinsfile লিখুন (Declarative), Docker agent-এ চালান, এবং GitHub Actions সংস্করণের সঙ্গে তুলনা করুন — কোনটি কোথায় ভালো, কোথায় খারাপ।

জমা: Jenkinsfile, Stage View-এর বিবরণ ও তুলনা-নোট।

মূল্যায়ন: Pipeline চলে কি না (৪০%), stage বিভাজন স্পষ্ট কি না (৩০%), তুলনা সৎ ও নির্দিষ্ট কি না (৩০%)।

অ্যাসাইনমেন্ট ১৪ — নিরাপত্তা স্ক্যান প্রতিবেদন

কাজ: আপনার অ্যাপে `npm audit` ও Trivy scan চালান, ফলাফল পড়ে একটি প্রতিবেদন লিখুন — কোন দুর্বলতা গুরুতর, কোনটি নয়, কোনটি ঠিক করবেন, কোনটি গ্রহণ করবেন (accept) ও কেন।

জমা: Scan ফলাফলের সারাংশ + সিদ্ধান্ত-টেবিল (ঠিক করব / গ্রহণ করব / পর্যবেক্ষণ করব) + যুক্তি।

মূল্যায়ন: গুরুত্ব অনুযায়ী সাজানো হয়েছে কি না (৪০%), প্রতিটি সিদ্ধান্তের যুক্তি আছে কি না (৪০%), false positive নিয়ে সতর্কতা আছে কি না (২০%)।

অ্যাসাইনমেন্ট ১৫ — Staging Deploy ও Rollback মহড়া

কাজ: একটি ভাঙা সংস্করণ staging-এ deploy করুন, health check দিয়ে ধরুন, rollback করুন, এবং পুরো ঘটনার incident নোট লিখুন — কখন কী হলো, কত সময় লাগল, পরের বার কী বদলাবেন।

জমা: Incident নোট (সময়সহ), verify ও rollback script, এবং উন্নতির তালিকা।

মূল্যায়ন: Health check ভাঙা deploy ধরেছে কি না (৩০%), rollback সফল কি না (৩০%), incident নোট সম্পূর্ণ ও সৎ কি না (৪০%)।

মেজর প্রজেক্ট 1

প্রজেক্ট ১ — GitHub Actions CI System (Node.js)

লক্ষ্য: একটি Node.js (MERN) অ্যাপের জন্য সম্পূর্ণ CI ব্যবস্থা বানানো — pull request যাচাই, lint, test, build যাচাই, artifact, cache ও নিরাপদ permission-সহ। এই প্রজেক্টের শেষে আপনার রিপোজিটরিতে ভাঙা কোড merge হওয়া প্রায় অসম্ভব হবে।

প্রয়োজনীয়তা (Requirements)

- প্রতিটি PR-এ lint, unit test ও build চলবে; কোনোটি ব্যর্থ হলে merge বন্ধ থাকবে।
- Node 18, 20 ও 22-এ matrix test চলবে (PR-এ ছোট matrix, main-এ পূর্ণ matrix)।
- Dependency cache থাকবে; cache ছাড়া run-এর চেয়ে সময় স্পষ্ট কম হবে।
- Build artifact (`dist/`) ৭ দিনের retention-সহ upload হবে।
- Workflow-এর permission least privilege হবে — `contents: read`, অপ্রয়োজনীয় write নয়।
- Third-party action সব পিন করা সংস্করণে (`@v4` / নির্দিষ্ট commit) ব্যবহার হবে।

আর্কিটেকচার:

PR → Workflow: ci.yml → Job: lint+test (matrix 18/20/22) → Job: build (needs: test) → Artifact: app-build → Status check → Branch protection: merge বন্ধ যতক্ষণ না সবুজ

main-এ push → একই workflow + coverage artifact

সম্পূর্ণ কনফিগারেশন

```
# .github/workflows/ci.yml
name: CI
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
permissions: { contents: read }
jobs:
  test:
    strategy:
      fail-fast: false
      matrix: { node-version: ['18', '20', '22'] }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: ${{ matrix.node-version }}, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test
  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: { name: app-build, path: dist/, retention-days: 7 }
```

মাইলস্টোন ও গ্রহণযোগ্যতা (Milestones & Acceptance Criteria)

মাইলস্টোন	গ্রহণযোগ্যতার মানদণ্ড
১. PR যাচাই	ভাঙা PR-এ merge বন্ধ; ভালো PR-এ merge খোলে।
২. Matrix test	তিনটি Node সংস্করণে আলাদা ফল দেখা যায়।
৩. Cache ও artifact	দ্বিতীয় run-এ install সময় কমে; artifact ডাউনলোড করা যায়।
৪. নিরাপত্তা	সব action পিন করা; permission শুধু `contents: read`; secret log-এ নেই।

বাস্তবায়ন নথি

প্রতিটি মাইলস্টোনের পর একটি ছোট নোট লিখুন: কী করলেন, কী পরীক্ষা করলেন, কী ভেঙেছিল ও কীভাবে ঠিক করলেন। শেষে পুরো workflow-এর একটি চিত্র (ডায়াগ্রাম) আঁকুন ও README-তে “CI কীভাবে কাজ করে” — এক পৃষ্ঠায় লিখুন। এই নথিই প্রজেক্টের প্রমাণ — শুধু সবুজ টিক চিহ্ন নয়।

> **PRO TIP:** প্রজেক্ট শেষে ইচ্ছা করে একটি ভাঙা PR খুলে দেখান — merge বন্ধ থাকা দেখানোই এই প্রজেক্টের সবচেয়ে শক্তিশালী প্রমাণ।

মেজর প্রজেক্ট 2

প্রজেক্ট ২ — Multi-Platform CI/CD তুলনা

লক্ষ্য: একই MERN অ্যাপের একই pipeline তিনটি প্ল্যাটফর্মে বাস্তবায়ন করা — GitHub Actions, GitLab CI/CD ও Jenkins — এবং প্রমাণ-ভিত্তিক তুলনা লেখা। কোন প্ল্যাটফর্ম "সেরা" — এর কোনো সর্বজনীন উত্তর নেই; উত্তর নির্ভর করে টিম, বাজেট ও পরিচালনার ক্ষমতার উপর।

তিনটি বাস্তবায়ন (একই অ্যাপ, একই ধাপ)

সাধারণ ধাপ (সব প্ল্যাটফর্মে একই): checkout → Node 20 setup → `npm ci` → lint → unit test → build → artifact সংরক্ষণ। প্রতিটি প্ল্যাটফর্মে এই ধাপগুলো বাস্তবায়ন করুন — GitHub-এ `.github/workflows/ci.yml`, GitLab-এ `.gitlab-ci.yml`, Jenkins-এ `Jenkinsfile`। প্রতিটি বাস্তবায়নের run-এর সময় মাপুন (তিনবার করে, গড় নিন)।

```
# GitLab (.gitlab-ci.yml) — সংক্ষেপ (ল্যাব ১৪-এর পূর্ণ সংস্করণ দেখুন)
# stages: [test, build]
# test-job: { stage: test, script: [npm ci, npm run lint, npm test] }
# build-job: { stage: build, script: [npm run build], artifacts: { paths: [dist/] } }
# Jenkins (Jenkinsfile) — সংক্ষেপ (ল্যাব ১৯-এর পূর্ণ সংস্করণ দেখুন)
# pipeline { agent { docker { image 'node:20-alpine' } }
#   stages { stage('Test') { steps { sh 'npm ci && npm run lint && npm test' } }
#            stage('Build') { steps { sh 'npm run build' } } } }
```

তুলনা-টেবিল

মাপকাঠি	GitHub Actions	GitLab CI/CD	Jenkins
স্থাপত্য (Architecture)	GitHub-হোস্টেড; YAML workflow; runner GitHub-এর বা নিজের	GitLab-হোস্টেড; একটিই `.gitlab-ci.yml`; stage/job মডেল	নিজে হোস্ট করতে হয়; controller/agent; Jenkinsfile (Groovy)
রক্ষণাবেক্ষণ (Maintenance)	সবচেয়ে কম — GitHub আপডেট করে	কম — GitLab আপডেট করে; নিজের runner থাকলে তার যত্ন লাগে	সবচেয়ে বেশি — আপডেট, ব্যাকআপ, plugin, নিরাপত্তা নিজের
নিরাপত্তা (Security)	OIDC, environment, permission — বিল্ট-ইন; secret GitHub-এ	Protected variable, environment, OIDC — বিল্ট-ইন	শক্তিশালী কিন্তু নিজে কনফিগার করতে হয়; ভুল কনফিগারেশন ঝুঁকিপূর্ণ
খরচ (Cost)	Public repo-তে ফ্রি; private-তে ব্যবহার-ভিত্তিক (সীমা ও মূল্য GitHub-এর ডক দেখুন — এই বইয়ে মূল্য দেওয়া হয়নি)	Shared runner-সহ ফ্রি tier; নিজের runner-এর খরচ আলাদা	সফটওয়্যার ফ্রি; সার্ভার, বিদ্যুৎ ও পরিচালনার খরচ নিজের
ডেভেলপার অভিজ্ঞতা (DX)	PR-এর সঙ্গে গভীরভাবে মিশে আছে; Actions ট্যাবে log সহজ	Pipeline চিত্র ও environment এক জায়গায়; একটিই ফাইল	শক্তিশালী কিন্তু শেখার বক্ররেখা খাড়া; UI পুরনো ধাঁচের

পরিচালনা জটিলতা	কম	কম-মাঝারি	বেশি — বিশেষ করে বড় টিমে

আপনার মাপা ফলাফল: উপরের টেবিলের পাশে আপনার নিজের মাপা সময় যোগ করুন — প্রতিটি প্ল্যাটফর্মে গড় run সময়, ব্যর্থতা থেকে পুনরুদ্ধারের সময় ও ডিবাগে কত সময় লাগল। বইয়ের টেবিল সাধারণ নির্দেশনা; আপনার মাপই আপনার সিদ্ধান্তের ভিত্তি।

গ্রহণযোগ্যতা

তিনটি pipeline-ই একই অ্যাপে চলে ও সবুজ হয়; তুলনা-টেবিলের প্রতিটি ঘরে আপনার নিজের পরীক্ষার প্রমাণ আছে (শুধু মতামত নয়); শেষে একটি সুপারিশ লিখুন — একটি ৫ জনের startup, একটি ৫০ জনের কোম্পানি ও একটি ব্যাংক — তিনটি ক্ষেত্রে কোন প্ল্যাটফর্ম, এবং কেন।

মেজর প্রজেক্ট 3

প্রজেক্ট ৩ — TechNova Automated Delivery System

লক্ষ্য: TechNova-র MERN অ্যাপের জন্য সম্পূর্ণ স্বয়ংক্রিয় delivery ব্যবস্থা — Git সহযোগিতা থেকে production approval ও rollback পর্যন্ত। এটি এই বইয়ের সব ল্যাবের সমন্বয়।

প্রয়োজনীয়তা (Requirements)

- Git-ভিত্তিক সহযোগিতা: feature branch → PR → review → merge; `main` সুরক্ষিত।
- স্বয়ংক্রিয় CI: lint, test, build, coverage।
- Docker image build (multi-stage), GHCR-এ publish, সংস্করণ tag (`v*`) ও digest।
- নিরাপত্তা স্ক্যান: dependency audit ও image scan; critical দুর্বলতায় pipeline থামবে।
- Staging deploy — অনুমোদনসহ; production deploy — আলাদা অনুমোদনসহ।
- Deploy যাচাই (health check) ও rollback script।
- Release versioning (Semantic Versioning) ও release note।

আর্কিটেকচার:

Developer → PR → CI (lint/test/build/scan) → Merge to main → Tag v1.2.3 → Docker build → GHCR push → Staging deploy (approval) → Health check ✅ → Production deploy (approval) → Health check ✅ → Release note প্রকাশ

❌ যেকোনো ধাপে ব্যর্থতা → deploy থামে → Rollback (আগের tag)

মাইলস্টোন

মাইলস্টোন	কাজ	গ্রহণযোগ্যতা
১. ভিত্তি	PR workflow, branch protection, CI	ভাঙা PR merge হয় না
২. Artifact	Docker build + GHCR publish	Tag-এ image আসে, digest লেখা থাকে
৩. নিরাপত্তা	npm audit + Trivy scan	Critical দুর্বলতায় pipeline লাল
৪. Staging	Approval + deploy + health check	অনুমোদন ছাড়া deploy হয় না; health check সবুজ
৫. Production	আলাদা approval + deploy + rollback মহড়া	Rollback মহড়া সফল; incident নোট লেখা

বাস্তবায়ন নথি

একটি `docs/delivery.md` ফাইল লিখুন — এতে থাকবে: আর্কিটেকচার চিত্র, প্রতিটি workflow-এর উদ্দেশ্য, secret-এর তালিকা (নাম শুধু, মান নয়), environment-এর নিয়ম, release প্রক্রিয়া (tag কীভাবে দেবেন), rollback পদ্ধতি ও জরুরি যোগাযোগের ধাপ। এই নথি পড়ে একজন নতুন DevOps Engineer যেন প্রথম দিনেই deploy করতে পারে — সেটাই এই নথির পরীক্ষা।

> **REAL WORLD:** বাস্তব টিমে এই নথিই “runbook”। Pipeline ভেঙে গেলে প্রথমে কেউ কোড দেখে না — runbook দেখে। তাই নথি লেখা কোড লেখার মতোই গুরুত্বপূর্ণ কাজ।

ক্যাপস্টোন প্রজেক্ট

ক্যাপস্টোন — TechNova-র সম্পূর্ণ Delivery Platform

আপনার ভূমিকা: আপনি TechNova-র DevOps Engineer। TechNova-র MERN অ্যাপ (React frontend, Express API, MongoDB) এখন manual deploy-এ চলে — কেউ SSH করে deploy করে, কখনো test বাদ পড়ে, কখনো ভাঙা সংস্করণ production-এ যায়। আপনার কাজ: একটি সম্পূর্ণ delivery platform ডিজাইন ও বাস্তবায়ন করা, যা টিমের সবাই ব্যবহার করবে।

পরিস্থিতি: টিমে ৬ জন developer। প্রতিদিন ৫–১০টি PR আসে। Staging সার্ভার আছে, production সার্ভার আলাদা। Production deploy-এ ভুল হলে গ্রাহকরা সঙ্গে সঙ্গে টের পান। আপনার platform-এ ভুল ধরা পড়বে deploy-এর আগেই, আর ভুল deploy হলে ফিরে আসার পথ থাকবে।

বাধ্যতামূলক ফলাফল (Mandatory Outcomes)

#	ফলাফল	প্রমাণ
১	PR-ভিত্তিক workflow	Feature branch → PR → CI → review → merge; `main`-এ সরাসরি push বন্ধ
২	স্বয়ংক্রিয় build ও test	প্রতিটি PR-এ lint, unit test, build চলে
৩	Code quality gate	lint, coverage রিপোর্ট ও security scan — ব্যর্থতায় merge বন্ধ
৪	নিরাপদ pipeline credential	Secret কোডে নেই; least-privilege permission; OIDC ধারণা নথিতে লেখা
৫	Docker image তৈরি	Multi-stage Dockerfile; root নয়, `USER node`
৬	Registry publishing	GHCR-এ সংস্করণ tag ও digest-সহ image
৭	Staging deployment	Tag-এ staging deploy; health check সবুজ
৮	অনুমোদন-ভিত্তিক production deployment	Production environment-এ required reviewer; অনুমোদন ছাড়া deploy হয় না
৯	Health check	`/health` endpoint ও verify script; ৫ বার retry
১০	Rollback	Rollback script ও সফল মহড়ার incident নোট
১১	Release versioning	Semantic Versioning tag (`v1.2.3`) ও release note
১২	Pipeline documentation	`docs/delivery.md` — নতুন Engineer পড়ে deploy করতে পারে

সম্পূর্ণ আর্কিটেকচার:
Developer → PR → CI (lint/test/build/scan, matrix) → Review + Merge → Tag v* → Docker build → Trivy scan → GHCR → Staging (approval → deploy → health check) → Production (approval → deploy → health check) → Release note
যেকোনো ধাপে ❌ → deploy থামে → Rollback (আগের tag + health check)

কাজের ধাপ (সুপারিশকৃত ক্রম)

ধাপ ১: রিপোজিটরি ও branch সুরক্ষা বসান (ল্যাব ২–৩)। **ধাপ ২:** CI workflow লিখুন — lint, test, build, matrix, cache, artifact (ল্যাব ২, ৫, ৯–১১; প্রজেক্ট ১)। **ধাপ ৩:** Security scan যোগ করুন (ল্যাব ২২)। **ধাপ ৪:** Dockerfile ও GHCR publish (ল্যাব ২১)। **ধাপ ৫:** Staging deploy — approval ও health check (ল্যাব ২৩–২৪)। **ধাপ ৬:** Production deploy — আলাদা approval (ল্যাব ১২, ২৩)। **ধাপ ৭:** Rollback মহড়া করুন ও incident নোট লিখুন (ল্যাব ২৪; অ্যাসাইনমেন্ট ১৫)। **ধাপ ৮:** `docs/delivery.md` লিখুন ও একজন সহপাঠীকে দিয়ে পড়িয়ে দেখুন — সে আপনার নথি পড়ে deploy করতে পারে কি না।

> **সতর্কতা:** Capstone-এর solution guide নিচে দেওয়া আছে — কিন্তু প্রথমে নিজে চেষ্টা করুন। Solution পড়ার আগে অন্তত ধাপ ১–৪ নিজে শেষ করুন। Solution দেখে কপি করলে আপনি pipeline লিখতে শিখবেন না — শুধু কপি করতে শিখবেন।

গ্রেডিং রুব্রিক (Grading Rubric — মোট ১০০)

মানদণ্ড	ওজন	পূর্ণ নম্বরের শর্ত (পরিমাপযোগ্য)	আংশিক নম্বর
PR workflow ও branch সুরক্ষা	১০	ভাঙা PR-এ merge বন্ধের প্রমাণ আছে; `main` -এ সরাসরি push বন্ধ	CI আছে কিন্তু protection নেই — ৪
Build, test ও quality gate	১৫	lint + test + build সব PR-এ চলে; কোনো gate ব্যর্থ হলে merge বন্ধ	একটি gate অনুপস্থিত — ৮
Matrix, cache ও artifact	১০	৩টি Node সংস্করণে test; cache-এ সময় কমার মাপ আছে; artifact ডাউনলোড করা যায়	একটি অনুপস্থিত — ৫
নিরাপদ credential ও permission	১৫	কোডে কোনো secret নেই; permission least privilege; সব action পিন করা; OIDC নথিতে ব্যাখ্যা আছে	একটি ঘাটতি — ৭
Docker image ও registry	১০	Multi-stage build; GHCR-এ tag ও digest-সহ image; non-root user	Single-stage বা `latest` -শুধু — ৫
Staging ও production deploy	১৫	দুটি environment-ই approval-সহ; staging deploy সফল; production deploy অনুমোদনে চলে	একটি environment-এ approval নেই — ৭
Health check ও rollback	১৫	Health check ভাঙা deploy ধরে; rollback মহড়া সফল ও incident নোট লেখা	Rollback মহড়া হয়নি — ৫
	১০		নথি আছে কিন্তু যাচাই হয়নি — ৫

Versioning ও documentation		SemVer tag ও release note আছে; `docs/delivery.md` পড়ে নতুন Engineer deploy করতে পারে (সহপাঠী যাচাই করেছে)	

গ্রেড: ৯০–১০০: Distinction — production-প্রস্তুত; ৭৫–৮৯: Merit — ছোট ঘাটতি আছে; ৬০–৭৪: Pass — মূল chain কাজ করে; ৬০-এর নিচে: Not yet — রুব্রিকের ঘাটতি পূরণ করে আবার জমা দিন।

ক্যাপস্টোন — সমাধান নির্দেশিকা (Solution Guide)

Capstone Solution Guide — প্রথমে নিজে চেষ্টা করুন, তারপর পড়ুন

এই অংশটি instructor-ধাঁচের সমাধান-রূপরেখা। Capstone-এর চেষ্টা শেষ করার পর এটি পড়ুন — নিজের সমাধানের সঙ্গে মিলিয়ে দেখুন, কপি করার জন্য নয়। প্রতিটি অংশে "কেন" ব্যাখ্যা করা আছে — কারণ একই সমস্যার একাধিক সঠিক সমাধান আছে।

১. PR Workflow ও Branch সুরক্ষা (সমাধান-রূপরেখা)

`main`-এ branch protection: "Require a pull request before merging" + "Require status checks to pass" (check: test ও build job) + "Require branches to be up to date"। সরাসরি push বন্ধ — এতে কেউ CI এড়িয়ে কোড ঢোকাতে পারে না। কেন "up to date" দরকার: পুরনো branch-এর PR merge করলে main-এর নতুন কোডের সঙ্গে সংঘাত (conflict) ধরা পড়ে না।

২. CI Workflow (সমাধান-রূপরেখা)

```
# .github/workflows/ci.yml — capstone-এর মূল workflow (প্রজেক্ট ১-এর সম্প্রসারণ)
name: CI
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
permissions: { contents: read }
jobs:
  test:
    strategy: { fail-fast: false, matrix: { node-version: ['18', '20', '22'] } }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: ${{ matrix.node-version }}, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test
  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: npm }
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: { name: app-build, path: dist/, retention-days: 7 }
```

কেন এই গঠন: test ও build আলাদা job — test ব্যর্থ হলে build-এর সময় নষ্ট হয় না; matrix-এ `fail-fast: false` — কোন সংস্করণে ভাঙে, তা জানা যায়; permission শুধু `contents: read` — CI-এর write দরকার নেই।

৩. Security Scan (সমাধান-রূপরেখা)

আলাদা workflow `security.yml` (ল্যাব ২২): `npm audit --audit-level=high` + Trivy image scan (HIGH, CRITICAL)। কেন আলাদা workflow: scan ধীর হতে পারে — CI-এর মূল পথে না রেখে সমান্তরালে চালালে PR-এর অপেক্ষা কমে; তবে merge-এর আগে scan-ও সবুজ হতে হবে — branch protection-এ scan job-কেও required check হিসেবে যোগ করুন।

৪. Docker ও Registry (সমাধান-রূপরেখা)

Multi-stage Dockerfile (ল্যাব ২১): build stage-এ dev dependency, runtime stage-এ শুধু production dependency; `USER node`; tag: `v1.2.3` + `latest` নয় — deploy-এ digest ব্যবহার করুন। কেন digest: tag বদলে যেতে পারে, digest বদলায় না — rollback-এ ঠিক কোন image চলছিল, তা নিশ্চিত থাকে।

৫. Staging ও Production Deploy (সমাধান-রূপরেখা)

```
# .github/workflows/deploy.yml — রূপরেখা
# on: push tags ['v*']
# jobs:
#   deploy-staging: environment: staging (required reviewers: 1)
#     steps: docker pull (tag) → deploy → ./scripts/verify.sh
#   deploy-prod: needs: deploy-staging, environment: production (required reviewers: 1)
#     steps: docker pull (digest) → deploy → ./scripts/verify.sh
#   rollback: (workflow_dispatch, input: previous tag) → ./scripts/rollback.sh
```

কেন দুটি আলাদা approval: staging-এ ভুল ধরা পড়লে production বাঁচে; production-এ আলাদা reviewer — deploy করা ব্যক্তি নিজে অনুমোদন দিতে পারে না। কেন production-এ digest: staging-এ যাচাই করা ঠিক একই image production-এ যাবে — tag-এর আড়ালে image বদলে যাওয়ার সুযোগ নেই।

৬. Health Check, Rollback ও নথি (সমাধান-রূপরেখা)

`/health` endpoint শুধু "চালু আছি" নয় — ডেটাবেস সংযোগও পরীক্ষা করবে (MongoDB-তে `ping`)। Rollback script আগের tag parameter হিসেবে নেয় (ল্যাব ২৪)। `docs/delivery.md`-এ থাকবে: আর্কিটেকচার চিত্র, workflow-এর তালিকা, secret-এর নাম (মান নয়), environment নিয়ম, release ধাপ (tag → staging → production), rollback ধাপ ও জরুরি যোগাযোগ। নথির পরীক্ষা: একজন সহপাঠী শুধু নথি পড়ে staging deploy করতে পারলে নথি পাস।

> **PRO TIP — নিজের সমাধান মেলানো:** আপনার সমাধান এই রূপরেখার সঙ্গে হুবহু না মিললেও ঠিক হতে পারে — রুব্রিকের প্রতিটি ফলাফল (outcome) যাচাই করুন। কোনো outcome-এর প্রমাণ দিতে না পারলে সেখানেই ঘাটতি — সেটি পূরণ করুন, তারপর আবার জমা দিন।